



Carreras de Ingeniería Electrónica e Ingeniería en Computación

Informe del Trabajo de Graduación

Desarrollo de un dispositivo de seguimiento ocular como herramienta de apoyo académico para personas con movilidad reducida

Autores:

Natalia Carolina Borbón

Facundo Ordóñez Nadra

Directores:

Federico Marcelo Vargas

Ezequiel Musa

Jurados:

Esteban Daniel Volentini

Martín Gustavo Juárez

Luis Roque Di Pinto

*Este trabajo fue realizado en la Ciudad de San Miguel de Tucumán,
entre noviembre de 2025 y agosto de 2026.*

Resumen

Este trabajo desarrolló L.E.V.I., un dispositivo de bajo costo de seguimiento ocular, pensado para que estudiantes con limitaciones motrices puedan conectarse al sistema y utilizarlo tanto para rendir evaluaciones de opción múltiple como para comunicarse con su entorno educativo. El dispositivo está compuesto por dos módulos basados en microcontroladores económicos y opera de manera autónoma, generando su propia red inalámbrica sin necesidad de conexión a internet. A través del mismo, los docentes administran los cuestionarios, los tutores realizan el seguimiento académico del alumno, y los compañeros se comunican con él.

Su valor principal es ofrecer una alternativa considerablemente más económica que las soluciones comerciales existentes y, a diferencia de estas, diseñada específicamente para el ámbito académico. Nuestra propuesta amplía la autonomía y participación de estos estudiantes en las aulas.

En la memoria se detallan el proceso de selección del método de detección de la mirada, el diseño y la configuración de la red inalámbrica del sistema, el desarrollo del firmware, de la base de datos y de la interfaz web, el diseño y fabricación de las placas de circuito propias, y las mediciones de consumo y autonomía realizadas, junto con las conclusiones y trabajos futuros propuestos, aplicando contenidos de sistemas embebidos, redes, diseño de circuitos impresos, bases de datos y desarrollo web correspondientes a las carreras de Ingeniería electrónica e Ingeniería en computación.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen	I
1. Introducción General	1
1.1. Introducción	1
1.2. Estado del arte	1
1.3. Motivación	2
1.4. Alcance y objetivos	3
2. Introducción Específica	4
2.1. Seguimiento Ocular (Eye-tracking)	4
2.2. Procesamiento de Imágenes y Visión por Computadora	4
2.2.1. Conceptos fundamentales y biblioteca OpenCV	5
2.2.2. Coincidencia de patrones (Template Matching)	5
2.3. Redes Neuronales Artificiales	5
2.3.1. Definición y conceptos básicos	5
2.3.2. Principio de funcionamiento	6
2.3.3. Tipos y arquitecturas de redes	6
2.4. Sistemas Embebidos y Control Gráfico	7
2.4.1. Microcontroladores y microprocesadores	7
2.4.2. Familia de microcontroladores ESP32	8
2.4.3. Interfaces gráficas embebidas y biblioteca LVGL	8
2.5. Redes y Comunicaciones	9
2.5.1. Redes locales Wi-Fi	9
Direccionamiento IP y transmisión de datos	10
2.6. Arquitectura y Patrones de Software	11
2.6.1. Modelo Cliente/Servidor	11
2.6.2. Patrón de diseño Modelo-Vista-Controlador (MVC)	11
2.6.3. Arquitectura de una aplicación web	12
Frontend	12
Backend	14
2.7. Bases de Datos y Persistencia	14
2.7.1. Conceptos de bases de datos relacionales	14
2.7.2. Motores de bases de datos embebidos	15
3. Diseño e implementación	16
3.1. Solución propuesta	16
3.1.1. Descripción general del sistema	16
3.1.2. Tecnologías a utilizar	17
3.2. El detector ocular	22
3.2.1. Procesamiento local	22
3.2.2. Procesamiento remoto	29
3.3. Intercomunicación de hardware	34

3.3.1.	Clientes y servidores	34
3.3.2.	Servidor Wi-Fi local (Naty)	35
3.3.3.	Transmisión de datos (Naty)	35
3.4.	Base de datos (Naty)	35
3.5.	Aplicación web	35
3.5.1.	Frontend	35
3.5.2.	Backend (Naty)	41
3.6.	Interfaz de usuario táctil	41
3.7.	Alimentación	42
3.7.1.	Soluciones propuestas	43
3.7.2.	Descripción del sistema elegido	45
3.7.3.	Recarga y protección	45
3.7.4.	Estabilización de voltaje	48
3.8.	Costo total de desarrollo	54
4.	Ensayos y resultados	55
4.1.	Montaje	55
4.1.1.	Conexiones y PCB	55
4.1.2.	Ensamblaje del prototipo	56
4.2.	Pruebas de Software	56
4.2.1.	Pruebas del tracker	56
4.2.2.	Pruebas unitarias	56
4.3.	Pruebas de Hardware	56
4.4.	Pruebas de rendimiento	56
4.5.	Pruebas de integración	56
5.	Conclusiones	58
5.1.	Resultados obtenidos	58
5.2.	Comparación con productos similares	58
5.3.	Trabajo futuro	59

Índice de figuras

1.1. Tobii Glasses	2
2.1. Representación gráfica del Eye-Tracking	4
2.2. El proceso del <i>Machine Learning</i>	6
2.3. Diagrama simplificado de una red neuronal	6
2.4. Placa de desarrollo <i>Arduino Uno</i>	8
2.5. Placa de desarrollo <i>ESP32-CYD</i> (vista frontal y trasera)	9
2.6. Red Wi-Fi con un microcontrolador ESP32 actuando como punto de acceso	10
2.7. El sistema <i>Cliente/Servidor</i>	11
2.8. Patrón de diseño Modelo-Vista-Controlador (MVC)	12
2.9. Características de un componente web	13
3.1. Diagrama de bloques general del sistema	16
3.2. Diagrama de bloques del sistema específico	17
3.3. Algunas placas de desarrollo de la familia ESP32	18
3.4. Placa de desarrollo <i>ESP32-S3</i>	18
3.5. Cámara <i>OV2640</i> en distintas formas y ángulos de visión	20
3.6. Funcionamiento del protocolo SPI	21
3.7. Entornos de desarrollo (<i>Visual Studio Code, PlatformIO, Arduino IDE y Google Colab</i>)	22
3.8. Pantalla de creación de modelos de <i>Edge Impulse</i>	23
3.9. Reconocimiento de vegetales con <i>EdgeImpulse</i>	23
3.10. Entorno de <i>Google Colab</i>	24
3.11. Una muestra de las imágenes utilizadas para el dataset	25
3.12. Planilla con etiquetado de imágenes	25
3.13. Resultados del modelo desarrollado en <i>Google Colab</i>	26
3.14. Distintas partes del ojo y su nivel de oscuridad	27
3.15. Detección de rostro con <i>MediaPipe</i>	29
3.16. Facciones del rostro en <i>MediaPipe</i>	29
3.17. Facciones del rostro en <i>WebGazer</i>	30
3.18. Coeficiente de correlación normalizado con media cero.	33
3.19. Diagrama de pines del módulo <i>TP4056</i>	46
3.20. Corriente vs voltaje de carga	47
3.21. Circuito típico de aplicación del <i>TP4056</i>	48
3.22. Circuito elevador Boost	49
3.23. Circuito interno del chip <i>MT3608</i>	50
3.24. Gráfico explicativo del comportamiento del amplificador de error	50
3.25. Modulación por ancho de pulso (PWM)	51
3.26. Diagrama de pines del módulo <i>MT3608</i>	51
3.27. Regulación de carga del <i>MT3608</i>	52
3.28. Frecuencia vs Tensión de entrada en el chip <i>MT3608</i>	52
3.29. Eficiencia vs Tensión de entrada en el chip <i>MT3608</i>	53
3.30. Circuito comercial de los módulos <i>MT3608</i>	53

4.1. Esquema de conexiones para la PCB del circuito capturador	55
4.2. Esquema simplificado de la PCB del circuito capturador	56
4.3. Esquema simplificado de la PCB del circuito servidor	56
4.4. Layout de la PCB del circuito capturador	57
4.5. Layout de la PCB del circuito servidor	57

Índice de Tablas

3.1. Sistemas de comunicación inalámbrica	21
---	----

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción General

En este capítulo se presenta el dispositivo de seguimiento ocular desarrollado. Se detallan la motivación que impulsó el proyecto, el estado actual de esta tecnología, así como también los objetivos planteados y el alcance definido para el sistema.

1.1. Introducción

En el ámbito educativo, los estudiantes con movilidad reducida enfrentan dificultades tanto al momento de ser evaluados como para comunicarse con docentes, tutores y compañeros. Esta situación limita su autonomía y participación en las clases, principalmente porque las instituciones no cuentan con herramientas adaptadas a sus posibilidades físicas.

Frente a esto, la tecnología de seguimiento ocular, *eye-tracking*, se presenta como una alternativa práctica la cual permite interactuar con una pantalla utilizando únicamente la mirada, eliminando la necesidad de recurrir al movimiento de los brazos o a la asistencia de terceros.

Sin embargo, los sistemas de seguimiento ocular comerciales cuentan con dos grandes inconvenientes: tienen costos económicos muy elevados y no están enfocados en la dinámica educativa. En la práctica, no existen soluciones accesibles que permitan a los profesores armar y tomar exámenes y que, al mismo tiempo, le sirvan al alumno para comunicarse dentro del ámbito de estudio.

Para dar respuesta a esta necesidad, este trabajo presenta el desarrollo de un dispositivo de seguimiento ocular de bajo costo diseñado para que un estudiante con movilidad reducida pueda resolver cuestionarios de opción múltiple y comunicarse con su entorno académico.

1.2. Estado del arte

La tecnología del *eye-tracking* no es una novedad. Desde principios de siglo hasta la actualidad han existido múltiples empresas y desarrolladoras que implementan la detección ocular para dispositivos cotidianos. El caso más conocido es el de la empresa **Tobii AB**, dedicada al desarrollo de dispositivos con esta tecnología. Entre sus productos podemos encontrar los *Tobii Glasses* (anteojos con seguimiento ocular para experiencia de usuario, figura 1.1), y los *Tobii Pro Fusion* y *Tobii Pro Spectrum*, utilizados principalmente para la investigación.



Figura 1.1: Tobii Glasses

Por otro lado, la empresa **EyeComTec** desarrolla productos exclusivamente de software como *ECTKeyboard*, *ECTMouse* y *ECTTracker*, que se utilizan para manipular computadoras con tan solo el movimiento de los ojos.

En cuanto a la empresa **LC Technologies**, cuenta con el sistema *Eyegaze*, un dispositivo con hardware y software propios diseñado para realizar múltiples tareas como navegar por internet, reproducir contenido multimedia, redactar mensajes y controlar dispositivos externos.

También se puede mencionar a *EyeHarp*, un instrumento musical digital que permite tocar música mediante el movimiento de los ojos. Funciona principalmente con rastreadores oculares *Tobii*, aunque también puede utilizarse con otros dispositivos que controlan el puntero del mouse. Está diseñado especialmente para personas con discapacidades físicas o motrices, ofreciendo una forma inclusiva de expresión musical.

A partir de todos estos desarrollos podemos inferir que la tecnología del *eye-tracking* se encuentra sumamente avanzada en el año en el que se realiza este trabajo. Sin embargo, nuestro proyecto se enfoca exclusivamente en el ámbito educativo, y presenta algunas ventajas en relación a estos dispositivos, que se comentarán más adelante.

1.3. Motivación

Todos los ejemplos de sistemas que utilizan *eye-tracking* expuestos anteriormente, ya sea por hardware, software o una combinación de ambos, son herramientas de calidad que pueden aplicarse en una gran variedad de campos. Han aportado una amplia gama de dispositivos versátiles que ayudan tanto a investigaciones científicas y de marketing, como también a facilitar acciones cotidianas. Sin embargo, el principal problema de estos dispositivos es su alto costo económico. El uso de tecnologías de tan alto nivel implica a su vez altos costos de desarrollo, lo que tiene como consecuencia que muchos de ellos no son accesibles para usuarios en una situación económica desfavorable. Otro detalle que se pudo observar durante la investigación es que, a pesar de la versatilidad de funcionamiento de estos dispositivos, ninguno está centrado exclusivamente en el ámbito educativo, por lo que se necesita una alternativa más optimizada, intuitiva y sencilla para aplicar en este campo.

Por todo lo anterior expuesto es que se propone este proyecto. Una solución que mantenga la ergonomía y la portabilidad de los dispositivos antes mencionados, sumando a su vez un bajo costo económico, impulsado por el uso de hardware y software de bajos recursos y una orientación exclusiva al ámbito educativo de personas con discapacidades motoras. Este dispositivo tiene la intención de que pueda ser utilizado en instituciones educativas y familias de escasos recursos,

garantizando una accesibilidad mucho mayor a la que cualquier otro dispositivo de alta tecnología pueda ofrecer.

1.4. Alcance y objetivos

El objetivo general de este proyecto es desarrollar un dispositivo de bajo costo basado en micro-controladores económicos que permita a personas con movilidad reducida rendir evaluaciones de opción múltiple y comunicarse mediante seguimiento ocular (*eye-tracking*), funcionando de forma autónoma y sin conexión a internet (*offline*).

Para alcanzar esta meta, se plantean los siguientes objetivos específicos:

- Diseñar un dispositivo económico, ergonómico y portable, que funcione sin necesidad de acceso a internet.
- Brindar una interfaz intuitiva y de fácil uso, accesible desde cualquier dispositivo con navegador web.
- Permitir que los profesores gestionen cuestionarios de opción múltiple personalizados.
- Permitir que los tutores realicen un seguimiento académico integral del alumno.
- Facilitar un canal de comunicación directo para que docentes, tutores y compañeros puedan realizarle preguntas e interactuar con el estudiante.
- Implementar un sistema de registro e inicio de sesión de usuarios, con una base de datos local para el almacenamiento de usuarios, cuestionarios y respuestas.
- Realizar el procesamiento de la imagen del ojo del usuario mediante un microcontrolador embebido en el dispositivo.

En cuanto al alcance, el proyecto comprende el desarrollo integral de un dispositivo autónomo capaz de capturar la imagen ocular del alumno, procesar la dirección de la mirada y traducir esa información en acciones concretas sobre una interfaz web. El sistema está delimitado para operar en un entorno local, sin depender de conexión a internet ni de infraestructura de red externa, permitiendo el acceso a la plataforma mediante cualquier navegador web compatible.

Respecto a los usuarios del sistema, se delimitan los siguientes roles y permisos:

- Alumno: interactúa mediante la mirada para resolver las evaluaciones de opción múltiple y responder a preguntas informales del entorno.
- Profesor: encargado de crear, editar y administrar sus cuestionarios académicos.
- Tutor: responsable de consultar todos los cuestionarios, junto con las respuestas y los resultados obtenidos por el alumno.
- Compañero: habilitado para realizar preguntas esporádicas de opción múltiple con el fin de comunicarse con el alumno, las cuales no quedan registradas en el sistema.

Capítulo 2

Introducción Específica

En este capítulo se explorarán en términos muy generales las distintas tecnologías, arquitecturas, paradigmas y técnicas que resultaron útiles para el desarrollo del trabajo.

2.1. Seguimiento Ocular (Eye-tracking)

El *eye-tracking*, que se traduce literalmente como *seguimiento ocular*, es el proceso tecnológico que permite medir y registrar el punto de la mirada de una persona o el movimiento de sus ojos en relación con la cabeza. Para ello, comúnmente se utiliza como punto de referencia el centro de la pupila. A diferencia del seguimiento facial (que analiza músculos, cejas o expresiones), el *eye-tracking* se concentra estrictamente en la forma y movimientos del globo ocular (Fig. 2.1).

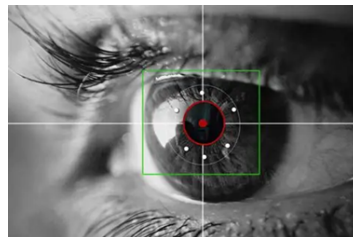


Figura 2.1: Representación gráfica del Eye-Tracking

Si bien esta tecnología cuenta con décadas de investigación y ha sido ampliamente utilizada en laboratorios de psicología, neuromarketing y análisis de usabilidad, el desarrollo de hardware más accesible ha permitido expandir sus aplicaciones. Una de estas aplicaciones directas es la *interacción basada en la mirada* (*gaze interaction*), donde los datos del movimiento ocular se utilizan para manipular un entorno digital.

En el caso del presente trabajo, orientado a la inclusión en el ámbito educativo, se implementa el seguimiento ocular enfocado en detectar hacia cuál de las opciones disponibles está dirigiendo la mirada el alumno. Esta información se procesa para registrar la selección correspondiente, permitiéndole responder cuestionarios de opción múltiple sin necesidad de manipular un cursor en pantalla ni requerir destreza física.

2.2. Procesamiento de Imágenes y Visión por Computadora

Para poner en práctica la disciplina del *Eye-Tracking* existen muchas maneras. En esta sección se describirá una técnica de visión computacional llamada *Coincidencia de patrones* o *Template Matching*, y *OpenCV*, una librería fundamental para su implementación.

2.2.1. Conceptos fundamentales y biblioteca OpenCV

El procesamiento de imágenes puede definirse como el proceso mediante el cual se modifica una imagen de entrada para obtener otra distinta mediante algoritmos matemáticos (ya sea para aumentar o disminuir su resolución, recortarla o cambiar su paleta de colores, entre otros procedimientos). En este contexto, dichas modificaciones se realizan con el fin de acondicionar la imagen para posteriores análisis computacionales.

Por otro lado, la visión computacional es el análisis que realiza una máquina sobre una imagen (usualmente procesada con anterioridad), con el fin de extraer características, interpretarlas y aprender de ellas, simulando la percepción visual humana. El *Eye-Tracking*, descrito en secciones anteriores, es un claro ejemplo de un proceso de visión computacional.

Lograr que una máquina interprete de manera autónoma el comportamiento de las imágenes es una tarea compleja, motivo por el cual se emplean herramientas especializadas que facilitan el proceso. Una de las más destacadas es *OpenCV (Open Source Computer Vision Library)*, la biblioteca de código abierto más popular e importante en el campo del análisis visual. Fue creada por la empresa *Intel* y cuenta con más de 2500 algoritmos matemáticos optimizados que sirven tanto para el procesamiento de imágenes como para la visión computacional. Entre sus principales capacidades destacan la manipulación estructural, la detección de características geométricas (como bordes, esquinas o líneas), la coincidencia de patrones, la detección de objetos y el análisis de movimiento.

2.2.2. Coincidencia de patrones (Template Matching)

La coincidencia de patrones, conocida en inglés como *Template Matching*, es una técnica de visión computacional diseñada para buscar y encontrar la ubicación de una imagen pequeña (denominada plantilla o molde) dentro de una imagen de mayor tamaño.

El proceso funciona deslizando la plantilla píxel por píxel sobre la imagen principal, de izquierda a derecha y de arriba hacia abajo. En cada posición evaluada, el algoritmo utiliza una fórmula matemática para calcular un puntaje de similitud entre la plantilla original y el fragmento de la imagen principal que se está superponiendo en ese instante.

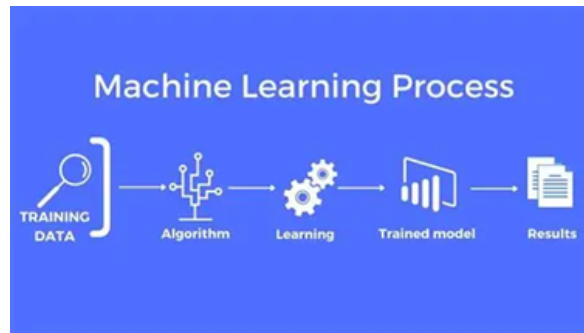
El resultado de esta operación no es una imagen, sino una matriz bidimensional de resultados. El punto con el valor estadístico más alto indica la coordenada exacta donde el algoritmo encontró la mayor coincidencia geométrica.

2.3. Redes Neuronales Artificiales

2.3.1. Definición y conceptos básicos

Para comprender este concepto es necesario definir previamente el *Machine Learning (ML)* o *Aprendizaje automático*. Esta rama de las ciencias de la computación permite a los sistemas aprender y resolver problemas sin ser programados de forma tradicional, suministrando datos a una computadora para que construya un modelo predictivo basado en esa experiencia (Fig. 2.2).

Dentro de los modelos de ML, uno de los más eficaces son las Redes Neuronales Artificiales. Reciben este nombre debido a que su estructura (basada en celdas o nodos interconectados para el procesamiento de información) se inspira directamente en el comportamiento biológico del sistema nervioso.

Figura 2.2: El proceso del *Machine Learning*

2.3.2. Principio de funcionamiento

Estas redes organizan sus neuronas en tres niveles jerárquicos principales: la capa de entrada (recibe los datos en crudo, como los píxeles de una imagen), las capas ocultas (el núcleo de procesamiento que detecta desde características básicas hasta patrones complejos) y la capa de salida (que entrega el resultado final, como las coordenadas de una pupila) (Fig. 2.3).

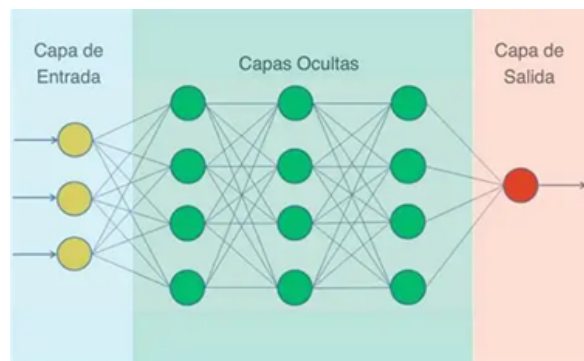


Figura 2.3: Diagrama simplificado de una red neuronal

El aprendizaje del modelo se logra mediante el entrenamiento, un proceso iterativo donde la red evalúa datos conocidos y ajusta su matemática interna para minimizar el margen de error. Durante estos ciclos, cada neurona ejecuta una ecuación fundamental:

$$z = \sum w_i x_i + b \quad (2.1)$$

Donde x_i representa el dato de entrada individual, w_i es el peso (la importancia relativa asignada a esa entrada específica) y b es el sesgo o bias (una constante de adaptación algorítmica). El resultado z es evaluado finalmente por una función de activación, la cual determina con qué intensidad la neurona transmite la señal hacia la siguiente capa.

2.3.3. Tipos y arquitecturas de redes

La arquitectura del modelo varía estrictamente según la naturaleza del problema a resolver. Se distinguen tres categorías fundamentales:

- Prealimentadas (*Feedforward*): La información viaja de forma unidireccional (de entrada a salida). Son ideales para problemas sencillos con datos livianos que no requieren memoria temporal.

- **Convolutionales (CNN):** Utilizan filtros matemáticos para analizar los datos espaciales por secciones. Son la arquitectura por excelencia para procesar imágenes y video en tiempo real.
- **Recurrentes (RNN):** Poseen bucles internos que les otorgan memoria secuencial, perfectas para contextos de análisis temporal (como procesamiento de lenguaje natural o secuencias largas de video).

Dado que este proyecto requiere analizar la posición espacial de las pupilas en un flujo de video en tiempo real, y considerando que se puede prescindir del contexto temporal (memoria secuencial), se concluye que la arquitectura a implementar en este dispositivo es una Red Neuronal Convolutacional.

2.4. Sistemas Embebidos y Control Gráfico

2.4.1. Microcontroladores y microprocesadores

En el diseño de sistemas embebidos actuales, la optimización de costos, espacio y consumo energético recae fundamentalmente en la implementación de microprocesadores y microcontroladores.

Un microprocesador (MPU) constituye la unidad central de procesamiento de un sistema. Se trata de un circuito integrado encargado de decodificar y ejecutar instrucciones aritméticas y lógicas. Su arquitectura interna se compone de una Unidad Aritmético Lógica (ALU), registros de memoria ultrarrápida y buses de datos y control que transmiten señales eléctricas binarias. Al carecer de memoria masiva y periféricos integrados, requiere de componentes externos interconectados en una placa base para conformar un sistema funcional.

Por otro lado, un microcontrolador (MCU) es un sistema informático completo encapsulado en un único circuito integrado. Su estructura integra un microprocesador central junto con bloques de memoria interna (tanto volátil para datos como no volátil para el programa) y múltiples periféricos. Estos módulos adicionales pueden incluir conversores analógico-digitales (ADC) y digital-analógicos (DAC), temporizadores, protocolos de comunicación serial y puertos de entrada/salida de propósito general (GPIO), lo que le permite interactuar directamente con el entorno físico y dispositivos externos de manera autónoma.

Un ejemplo sumamente conocido de microcontroladores es la placa de desarrollo *Arduino Uno* (Fig. 2.4)

Figura 2.4: Placa de desarrollo *Arduino Uno*

2.4.2. Familia de microcontroladores ESP32

Otro modelo de placas de desarrollo que ha ganado enorme popularidad en el desarrollo de sistemas embebidos y el Internet de las Cosas (IoT) es el microcontrolador ESP32. Creado por la compañía Espressif Systems, este dispositivo surgió como una solución de bajo costo y alta eficiencia energética diseñada para suplir la creciente necesidad de conectividad inalámbrica en proyectos electrónicos. A diferencia de las placas más tradicionales, el ESP32 destaca por integrar módulos de Wi-Fi y Bluetooth (tanto clásico como de baja energía) directamente dentro del mismo chip.

Además de sus excelentes capacidades de comunicación, este microcontrolador representa un salto significativo en cuanto a potencia computacional. Al estar generalmente equipado con un procesador de doble núcleo y una memoria considerablemente mayor que la de sus predecesores, permite manejar tareas más complejas, alojar servidores web locales y procesar datos a mayor velocidad. Gracias a este equilibrio entre rendimiento, conectividad y tamaño reducido, el ESP32 se ha convertido en una pieza fundamental para la ingeniería de dispositivos modernos e interconectados.

2.4.3. Interfaces gráficas embebidas y biblioteca LVGL

El desarrollo de interfaces gráficas de usuario (GUI, por sus siglas en inglés) en sistemas embebidos presenta desafíos asociados a las restricciones de memoria RAM, capacidad de cómputo y consumo de energía de los microcontroladores. A diferencia de los entornos convencionales, donde el sistema operativo gestiona los recursos visuales mediante unidades de procesamiento gráfico dedicadas (GPU), en un microcontrolador el renderizado debe ejecutarse de forma eficiente para garantizar fluidez visual y respuesta táctil en tiempo real sin sobrecargar la CPU.

Para resolver esta problemática, se recurre a bibliotecas gráficas optimizadas para hardware limitado, entre las que destaca **LVGL** (*Light and Versatile Graphics Library*). LVGL es una biblioteca de código abierto escrita en lenguaje C, diseñada para operar con requerimientos mínimos de memoria. Implementa un modelo orientado a objetos basado en elementos visuales predefinidos (*widgets*) como botones, etiquetas, listas, deslizadores y contenedores, los cuales pueden personalizarse mediante estilos y animaciones.

La arquitectura de LVGL se fundamenta en el desacoplamiento entre la lógica de la interfaz y el hardware subyacente. Esto se logra a través de dos componentes principales:

- Controlador de pantalla (*Display Driver*): transfiere los píxeles renderizados desde el búfer de dibujo interno de la biblioteca hacia el controlador físico del panel (por ejemplo, a través de interfaces SPI o paralelas), admitiendo técnicas de doble búfer (*double buffering*) para evitar parpadeos en pantalla.
- Controlador de dispositivo de entrada (*Input Device Driver*): gestiona la lectura y calibración de periféricos interactivos, como pantallas táctiles resistivas o capacitivas, teclados y codificadores rotativos.

En el ámbito del hardware comercial de bajo costo, esta combinación de pantalla, interfaz táctil y microcontrolador suele integrarse en módulos compactos denominados comúnmente *Cheap Yellow Display* (CYD), los cuales incorporan un SoC ESP32 junto con un panel TFT y un controlador táctil resistivo (típicamente XPT2046) interconectados mediante bus SPI. La integración de LVGL sobre estas plataformas permite ejecutar interfaces interactivas fluidas directamente en el dispositivo embebido de manera autónoma.



Figura 2.5: Placa de desarrollo *ESP32-CYD* (vista frontal y trasera)

2.5. Redes y Comunicaciones

2.5.1. Redes locales Wi-Fi

El estándar IEEE 802.11, conocido comúnmente como Wi-Fi, define los protocolos para la implementación de redes de área local inalámbricas (WLAN). La topología de estas redes se estructura a través de dos modos de operación:

- Modo Estación (*Station* - STA): en esta configuración, el nodo inalámbrico actúa como cliente. Su función consiste en explorar el espectro de radiofrecuencia, detectar una red existente a través de su identificador (SSID), autenticarse mediante los protocolos de seguridad correspondientes (como WPA2/WPA3) y asociarse a un punto de acceso central para integrarse al tráfico de dicha red.

- **Modo Punto de Acceso (*Access Point* - AP o SoftAP):** en este modo, el dispositivo asume el rol de concentrador e iniciador de la red. Es el encargado de emitir periódicamente tramas de sincronización (*beacon frames*) para anunciar la presencia de la red, autenticar a los nodos que solicitan conexión y gestionar la tabla de clientes asociados, de esta manera crea una infraestructura de red local independiente.

En el marco de este trabajo, uno de los módulos del sistema se configura en modo Access Point para generar una red local cerrada e independiente, mientras que tanto el módulo de captura de imagen como los dispositivos de los usuarios se conectan a dicha red en modo Station, permitiendo la comunicación directa entre todos los componentes sin depender de un router externo ni de acceso a internet.

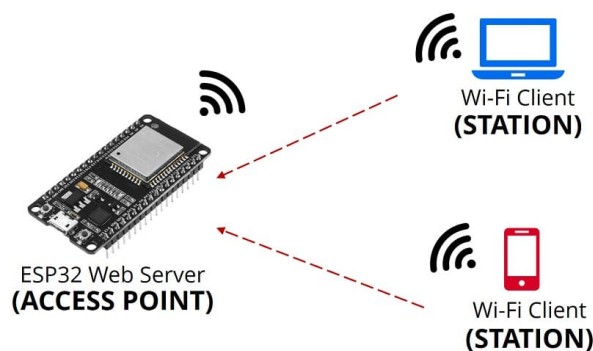


Figura 2.6: Red Wi-Fi con un microcontrolador ESP32 actuando como punto de acceso

Direccionamiento IP y transmisión de datos

La comunicación en una red de datos se rige por protocolos que definen cómo se estructura, transmite y recibe la información. Dentro de estas reglas, el direccionamiento lógico permite identificar a cada dispositivo de manera unívoca para asegurar que los datos alcancen su destino.

- **Protocolo IP (*Internet Protocol*):** estructura los datos en paquetes que contienen las direcciones de origen y destino para su entrega en la red. En redes locales bajo la versión IPv4, la dirección de cada dispositivo se establece mediante dos modalidades:
 - **Asignación estática:** la dirección se define de manera manual en el equipo y permanece fija a lo largo del tiempo, lo que permite localizar siempre al dispositivo en esa misma dirección.
 - **Asignación dinámica:** el protocolo de configuración dinámica de host (DHCP, *Dynamic Host Configuration Protocol*) asigna una dirección disponible de forma automática al asociarse el equipo a la red, lo que agiliza la incorporación de nuevos dispositivos sin requerir configuraciones individuales.
- **Protocolo HTTP (*Hypertext Transfer Protocol*):** opera bajo el modelo cliente-servidor mediante un esquema de peticiones y respuestas. Para asegurar que la información no sufra pérdidas ni alteraciones en su secuencia, HTTP utiliza TCP (*Transmission Control Protocol*), el cual establece una conexión confiable previa al intercambio de datos. Mediante este mecanismo se transfieren páginas web, estilos y datos estructurados.
- **Transmisión continua de video (*Streaming MJPEG*):** en lugar de realizar una petición HTTP por cada imagen, la técnica MJPEG (*Motion JPEG*) mantiene una única conexión

abierta para enviar una secuencia continua de fotogramas JPEG independientes delimitados por un separador. El cliente actualiza la imagen en pantalla a medida que recibe cada cuadro, logrando una visualización de video fluida en tiempo real.

2.6. Arquitectura y Patrones de Software

2.6.1. Modelo Cliente/Servidor

La arquitectura cliente/servidor (Fig. 2.7) es un modelo de diseño de software y redes en el cual las tareas a realizar se distribuyen entre un proveedor de recursos o servicios denominado “Servidor” y un receptor o demandante de dichos servicios denominado “Cliente”. Se basa en una dinámica de petición/respuesta entre ambas entidades. Sus componentes principales son:

- El **cliente**: Es el dispositivo o aplicación que interactúa con el usuario final. Su función principal es la de elaborar y enviar peticiones de datos o ejecución de tareas a través de una red, y luego recibir, formatear y presentar los resultados obtenidos en la interfaz de usuario. Es el componente que siempre inicia la comunicación.
- El **servidor**: Es el dispositivo o programa en ejecución continua diseñado para escuchar, recibir y procesar las peticiones de uno o múltiples clientes. Su función es la de administrar los recursos y servicios para devolver al cliente la respuesta solicitada.

Esta forma de trabajo es muy utilizada en los sistemas embebidos modernos para centralizar la carga de trabajo en un único dispositivo (el servidor), en lugar de que cada uno de los clientes tenga su propia base de datos y recursos, lo cual incrementaría significativamente los costos y la complejidad de hardware en cada terminal, volviendo los procesos ineficientes.

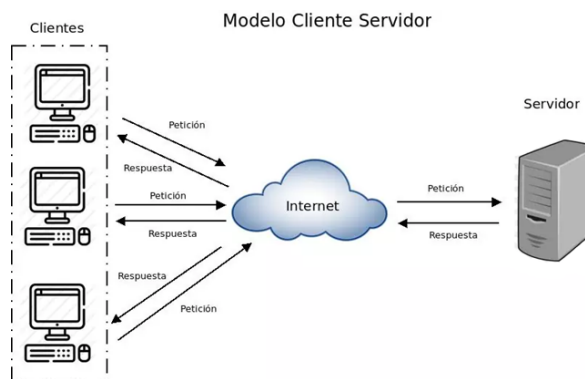


Figura 2.7: El sistema *Cliente/Servidor*

2.6.2. Patrón de diseño Modelo-Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador (*Model-View-Controller* - MVC) es una estrategia de diseño de software orientada a desacoplar la lógica de negocio, la gestión de datos y la interfaz de usuario en tres componentes con responsabilidades bien definidas:

- Modelo (*Model*): representa la estructura de los datos del sistema y gestiona las reglas de negocio. Se encarga de interactuar con los mecanismos de almacenamiento y persistencia para consultar, insertar, modificar o eliminar registros, asegurando la integridad de la información sin vincularse con la interfaz gráfica.

- Vista (*View*): constituye la capa de presentación con la que interactúa el usuario. Su función es recibir los datos procesados y renderizarlos en un formato comprensible, capturando los eventos y acciones del usuario para transferirlos al controlador.
- Controlador (*Controller*): actúa como intermediario entre el modelo y la vista. Procesa las solicitudes y eventos originados en la vista, ejecuta la lógica correspondiente, solicita las operaciones necesarias al modelo y actualiza la vista con el resultado.

Esta separación de responsabilidades favorece la modularidad, simplifica el mantenimiento del sistema y permite modificar la interfaz visual o el almacenamiento de datos de forma independiente sin alterar la lógica central de la aplicación.

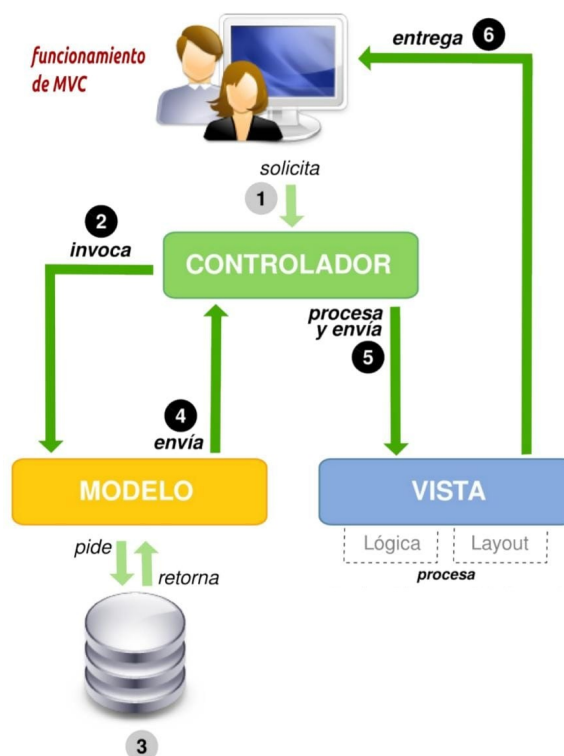


Figura 2.8: Patrón de diseño Modelo-Vista-Controlador (MVC)

2.6.3. Arquitectura de una aplicación web

En el desarrollo de software, las aplicaciones web suelen combinar el modelo cliente/servidor con el patrón MVC. Este enfoque distribuye las responsabilidades del sistema entre el cliente y el servidor, a los que se identifica como *frontend* y *backend* respectivamente. El *frontend* opera como la vista del sistema: presenta la interfaz gráfica, captura las acciones del usuario y emite las peticiones. El *backend* reúne las funciones del controlador y del modelo: atiende las solicitudes entrantes, procesa la lógica del sistema y administra la persistencia de los datos. A continuación, se detallan las características y la organización de cada una de estas partes.

Frontend

Al solicitar la aplicación, el navegador descarga un conjunto de archivos que procesa localmente para generar la interfaz. Este proceso se apoya en tres tecnologías complementarias:

- **HTML** (*HyperText Markup Language*, lenguaje de marcado de hipertexto): define la estructura y el contenido del documento. Mediante etiquetas, especifica qué función cumple cada elemento en la interfaz (títulos, bloques de texto, botones o casillas para ingresar datos), organizándolos en un orden lógico que el navegador interpreta para armar la pantalla.
- **CSS** (*Cascading Style Sheets*, hojas de estilo en cascada): controla el aspecto visual y la ubicación de los elementos definidos por HTML. Aplica reglas para definir colores, tipografías, tamaños y espacios, organizando cómo se distribuyen los bloques en la pantalla para que la vista se adapte bien según el tamaño del dispositivo.
- **JavaScript**: es el lenguaje de programación que se ejecuta directamente en el navegador. Mientras que el HTML define qué elementos existen y el CSS define cómo se ven, JavaScript es el encargado de que la página responda a las acciones del usuario: detecta eventos como clics o el envío de un formulario, altera el contenido en pantalla en tiempo real según esa interacción, y se comunica con el servidor para intercambiar información mientras la aplicación está en uso.

A medida que una aplicación suma pantallas y funciones, mantener el código fuente en archivos globales vuelve el sistema denso y difícil de ordenar. Para resolver esto, el desarrollo adopta una arquitectura basada en componentes (CBA, *Component-Based Architecture*). En este enfoque, la interfaz se estructura a partir de componentes independientes (Fig. 2.9), donde cada elemento funciona como una entidad autónoma que reúne su estructura, presentación y lógica asociada.

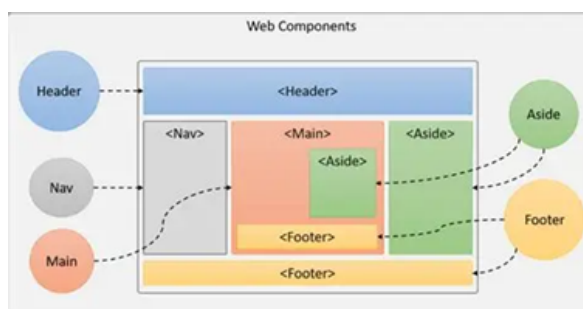


Figura 2.9: Características de un componente web

Bajo este modelo, el código fuente se divide en múltiples archivos modulares que se importan y articulan en un archivo principal. La única relación de dependencia entre componentes se da cuando uno contiene a otro. En esa situación, el componente hijo no puede alterar directamente los datos o estados del padre, sino que solo puede emitir peticiones para solicitar cambios, mientras que el padre sí puede modificar los estados del hijo. Esta estructura ofrece una fácil reusabilidad de elementos, además de garantizar la modularidad y el aislamiento de fallos; es decir, si un componente específico presenta un error, no compromete al funcionamiento del programa en su totalidad.

Para implementar este esquema se utiliza Preact, una biblioteca orientada a componentes que ofrece las mismas capacidades de desarrollo que React pero con una sobrecarga mucho menor. Al prescindir de funciones accesorias innecesarias, el empaquetado final resulta significativamente más liviano, lo que reduce el tamaño de los archivos que el microcontrolador debe almacenar y transferir hacia el navegador al momento de servir la interfaz.

Backend

En este desarrollo, el backend adopta una arquitectura en capas basada en el patrón Controlador-Servicio-Repositorio. Esta estructura deriva directamente de MVC ya que, al resolverse la Vista de forma independiente en el cliente web, los Controladores conservan su rol original mientras que el Modelo tradicional se divide en dos capas separadas para diferenciar la lógica de la persistencia de datos.

- Controladores (*controllers*): reciben las peticiones que llegan desde el cliente web, extraen los datos de la solicitud y llaman al servicio correspondiente para devolver luego la respuesta. No aplican reglas de funcionamiento ni interactúan con los datos guardados.
- Servicios (*services*): representan la parte del Modelo encargada de la lógica del sistema. Reciben los datos procesados por el controlador, aplican las reglas operativas, realizan validaciones y coordinan cuándo consultar o modificar información mediante los repositorios.
- Repositorios (*repositories*): representan la parte del Modelo dedicada exclusivamente a los datos y su almacenamiento. Son los únicos que leen y escriben en la base de datos o en la memoria, aislando los detalles de la persistencia del resto del sistema.

En este trabajo, estas capas se desarrollaron en C++ junto con el resto del código del sistema embebido, ejecutándose todo dentro del microcontrolador.

2.7. Bases de Datos y Persistencia

2.7.1. Conceptos de bases de datos relacionales

Una base de datos es una colección organizada e integrada de datos almacenados de forma persistente para su posterior consulta y manipulación. Dentro de los distintos paradigmas existentes, el modelo relacional organiza la información en estructuras bidimensionales denominadas tablas (o relaciones), compuestas por filas (registros o tuplas) y columnas (campos o atributos).

Los fundamentos de este modelo se basan en los siguientes conceptos:

- Estructura y esquemas: cada tabla posee un esquema definido que determina el tipo de dato que puede almacenar cada columna (numérico, texto, fecha, etc.), garantizando la consistencia del formato.
- Claves e integridad relacional: para identificar de forma unívoca cada registro se utiliza una clave primaria (*Primary Key*). La vinculación entre distintas tablas se establece mediante claves foráneas (*Foreign Keys*), lo que permite relacionar entidades sin duplicar información (integridad referencial).
- Lenguaje de consulta estructurado (SQL): es el lenguaje formal estándar (*Structured Query Language*) empleado para definir la estructura de los datos, realizar búsquedas mediante filtros, e insertar, modificar o eliminar información de manera declarativa.
- Transacciones y propiedades ACID: una transacción es un conjunto de operaciones que se ejecutan como una única unidad lógica. Para garantizar la confiabilidad, los sistemas relacionales implementan las propiedades ACID: Atomicidad (la operación se completa en su totalidad o no se aplica), Consistencia (se respetan todas las reglas e integridad del esquema), Aislamiento (las operaciones simultáneas no interfieren entre sí) y Durabilidad (los cambios confirmados persisten ante fallos o cortes de energía).

2.7.2. Motores de bases de datos embebidos

En el desarrollo de software existen dos formas principales de gestionar una base de datos: mediante sistemas basados en un servidor independiente (que atienden peticiones a través de una red) o mediante **motores embebidos**.

Un motor de base de datos embebido es una biblioteca de software que se integra directamente dentro del mismo código de la aplicación. En este esquema, el motor no se ejecuta como un proceso separado, sino que forma parte del propio ejecutable del sistema.

SQLite es un motor de base de datos relacional embebido de código abierto ampliamente utilizado que implementa el estándar SQL. Sus características teóricas esenciales son:

- **Arquitectura sin servidor (*Serverless*)**: los procesos leen y escriben directamente en el soporte de almacenamiento, eliminando la necesidad de configurar, administrar o mantener un servicio en segundo plano.
- **Almacenamiento en archivo único**: la base de datos completa (esquema, tablas, índices y datos) se almacena dentro de un único archivo en el sistema de archivos del dispositivo, lo que simplifica su copia, transporte y respaldo.
- **Autonomía y portabilidad (*Self-contained*)**: no depende de bibliotecas externas complejas ni de configuraciones previas en el sistema operativo, permitiendo su funcionamiento en entornos con recursos de memoria y procesamiento acotados.
- **Cumplimiento transaccional**: a pesar de su tamaño compacto, implementa transacciones atómicas que aseguran que los datos no se corrompan ante reinicios o pérdidas de alimentación eléctrica imprevistas.

Capítulo 3

Diseño e implementación

3.1. Solución propuesta

En esta sección se realizará una descripción general del sistema y las tecnologías que se utilizarán para este trabajo.

3.1.1. Descripción general del sistema

El hardware necesario para construir un dispositivo de *eye-tracking* se puede resumir en tres partes principales:

- Una **cámara** para captar movimientos oculares.
- Un **procesador** que cumpla múltiples funciones. Entre ellas, analizar la imagen, determinar la dirección hacia la que el usuario está mirando y correr una aplicación de software interactiva con estos movimientos.
- Una **pantalla o monitor** que se comuniqué con el procesador para mostrar la interfaz de la aplicación.

Para nuestro trabajo se necesita de una cuarta parte adicional, que engloba a todos los dispositivos periféricos necesarios para que los examinadores, compañeros o familiares puedan personalizar las preguntas que se le realizan al estudiante (se aclaró previamente que, junto al monitor, lo proveerán los dispositivos externos). A grandes rasgos, el sistema se resume en el diagrama de la figura 3.1.

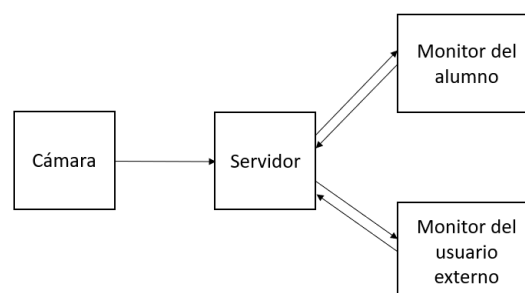


Figura 3.1: Diagrama de bloques general del sistema

En cuanto a los requisitos de software, se necesita que el dispositivo pueda almacenar información de usuarios, preguntas y respuestas en una base de datos propia, para que cualquier usuario registrado pueda informarse respecto al rendimiento académico del estudiante que lo utilizará. Una vez establecidos los requisitos mínimos, se describe a continuación cómo funcionará nuestro sistema.

El núcleo central de nuestro trabajo se basa en el uso de un microcontrolador de bajo costo encargado de realizar todo el procesamiento de imagen y aplicaciones. Naturalmente, sus capacidades de memoria y procesamiento son limitadas, por lo que, para repartir la carga, se optó por dividir este núcleo principal en dos: un microcontrolador se ocupará de todo el procesamiento de imagen, mientras que el otro almacenará y ejecutará la aplicación principal con los datos de seguimiento proporcionados por el primero.

Para realizar la detección ocular utilizaremos una cámara diminuta e integrada en el microcontrolador de procesamiento de imagen. Será este mismo el encargado de manipular la cámara y captar las imágenes que luego se procesarán. Este análisis se realizará mediante una red neuronal artificial que, al entrenarla con imágenes de distintas direcciones de mirada, será capaz de medir con precisión las coordenadas exactas de la pupila dentro de cualquier imagen captada por la cámara.

Una vez finalice la detección, los datos de coordenadas serán transmitidos al segundo procesador para que gestione la interacción entre los movimientos oculares del estudiante y el software. La idea es que este segundo dispositivo sea capaz de ejecutar una aplicación web, transmitiendo los datos al dispositivo móvil externo mediante protocolo Wi-Fi sin acceso a internet.

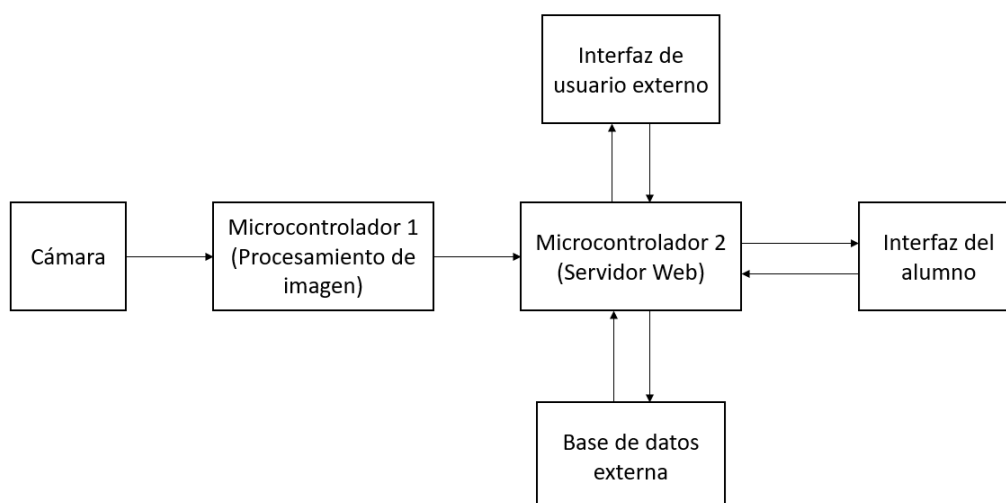


Figura 3.2: Diagrama de bloques del sistema específico

Utilizando todos estos componentes se logra que este dispositivo sea económico y portable, como se planteó en un inicio. Para además garantizar la ergonomía de uso, todo este sistema irá montado en la visera de una gorra común, para que la cámara pueda tener un buen ángulo del ojo a la vez de que se procura dar un uso cómodo y seguro.

(Imagen del montaje teórico?)

3.1.2. Tecnologías a utilizar

En esta sección se listan y justifican todos los componentes de hardware seleccionados para este proyecto, así como el rol que adoptará cada uno en el sistema expuesto en la sección anterior.

Microcontroladores

Los microcontroladores son el eje central del presente trabajo. Estos son los responsables de realizar la captura y procesamiento de imágenes, así como de la ejecución de la aplicación que

el usuario final visualizará en su monitor. Como se ha mencionado varias veces a lo largo de este informe, se busca desarrollar un dispositivo económico, por lo que los microcontroladores seleccionados deben ser de bajo costo y fácilmente accesibles. Además, se estableció que el microcontrolador que se utilizará para ejecutar la aplicación web debía poseer alguna forma de conexión WiFi.

En el mercado existen diversos modelos de procesadores que encajan con la descripción buscada, sin embargo, se optó por utilizar los módulos *ESP32-S3*, fabricados por la empresa china *Espressif Systems*.

La figura 3.3 muestra un conjunto de placas de desarrollo que utilizan microcontroladores ESP32, mientras que la figura 3.4 muestra el microcontrolador *ESP32-S3* seleccionado para este trabajo.

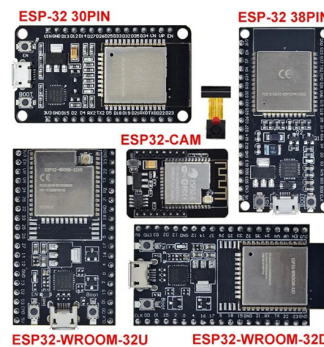


Figura 3.3: Algunas placas de desarrollo de la familia ESP32



Figura 3.4: Placa de desarrollo *ESP32-S3*

Para establecer un poco el contexto debemos describir sus modelos predecesores, remontándonos al año 2014 cuando salió al mercado el modelo *ESP8266*, una alternativa a las placas Arduino de uso general pero que incluían módulo de WiFi. Existían módulos Arduino con WiFi integrado pero no eran económicamente viables, lo que provocó que las placas de desarrollo de *Espressif* ganara popularidad rápidamente. En 2016 llegaría el *ESP-32* estándar que, aunque tenía un precio ligeramente más elevado (AR\$ 13.000 a valores actuales), sus características eran muy superiores al *ESP8266* (cuenta con más núcleos, más memoria, más periféricos, etc.). Hoy en día tenemos a disposición su variante *ESP32-S3*, el cual es mucho más potente, eficiente, veloz y cuenta con más espacio en memoria RAM que su versión clásica. Si bien su precio suele ser el doble (AR\$24.000 a valores actuales), todas estas características justifican el precio, ya que su potencia será necesaria para correr redes neuronales, transmitir datos y ejecutar aplicaciones web con relativa rapidez.

Cabe recordar que, como se estableció previamente en esta sección, se requiere de dos microcontroladores: uno para capturar y procesar la imagen y otro para servir como punto de acceso WiFi y ejecutar la aplicación web. En ambos casos se utilizará el procesador *ESP32-S3*. Por otro

lado, para capturar las imágenes se requiere una cámara y en el mercado existe una placa de desarrollo idéntica al *ESP32-S3* pero que cuenta con un módulo adicional para insertar cámaras pequeñas mediante un conector flex.

A continuación se listan algunas placas de desarrollo similares que se tuvieron en consideración y se justifica la razón de su descarte:

- *ESP8266* y *ESP32* clásico: Como se mencionó anteriormente, se descartan debido a que su memoria y velocidad de procesamiento no son adecuadas para los fines de este proyecto, que incluyen el procesamiento de imágenes y ejecución de redes neuronales.
- *Raspberry Pi Pico*: Una opción muy sólida, con características técnicas similares o incluso superiores a un *ESP32-S3* y con precios similares. El motivo por el que se descarta es el hecho de no contar con interfaz de cámara nativa, lo que obliga a usar cámaras con circuitos externos complejos, más lentos y que elevan el costo económico del proyecto original. Hay módulos Raspberry Pi que sí incluyen interfaz nativa para cámaras, pero su precio es mayor (>AR\$ 40.000).
- *Arduino Nano 33 IoT / Arduino Nano ESP32*: Además de contar con las mismas ventajas que el *ESP32-S3*, cuenta con el soporte robusto de la marca Arduino, pero se descarta porque sus costos son más elevados (entre 3 y 5 veces) que los de las placas de Espressif.

Además de estas marcas y modelos reconocidos, existen otras placas de desarrollo igual de económicas y eficientes que un *ESP32-S3*, pero estas, a diferencia de las desarrolladas por Espressif, tienen poca disponibilidad en los mercados regionales.

A continuación se listan las características técnicas de los *ESP32-S3*, los microcontroladores elegidos para este trabajo:

- CPU: Xtensa dual-core 32-bit LX7 DE HASTA 240 MHz
- Memoria: 384 KB ROM, 512 KB SRAM, 16 KB RTC SRAM
- GPIO: 45
- ADC: 2 × 12-bit SAR ADCs, hasta 20 canales
- WIFI AB: 20 MHz, 40 MHz in 2.4 GHz band
- Bluetooth 5
- Interfaces: 3 UART, 4 SPI, 2 I2C, 2 I2S, 1 RMT TX/RX
- Contadores y Timers: 1 Contador de pulsos, 3 Watchdog, 4 × 54-bit general, 1 × 52-bit sistema.
- PWM y USB: LED PWM hasta 8 canales, 2 MCPWM, 1 USB OTG, 1 USB Serial/JTAG

Cámara

Existen varios modelos de cámaras compatibles con la interfaz del *ESP32-S3*. En este trabajo no se exige una cámara de alta resolución, ya que el exceso de datos puede saturar la memoria del microcontrolador. Lo que se debe considerar son dos factores importantes: en primer lugar, la necesidad de una alta tasa de fotogramas por segundo (FPS). Esta velocidad es fundamental debido a que, en lugar de analizar imágenes estáticas, el sistema realiza un rastreo de coordenadas en tiempo real directamente desde el flujo de datos del sensor. Por otro lado, al estar el dispositivo montado en la visera de una gorra, el campo de visión (FOV) no debe ser muy

amplio, para evitar entorpecer a la red neuronal con estímulos ajenos a la pupila del ojo. Mediante el análisis empírico de imágenes con distintos ángulos, se determinó que para esta tarea es necesario un campo de visión de entre 60 y 120 grados.

En ese sentido, las cámaras *OV2640* de 2 Megapíxeles (frecuentemente incluidas en la compra de los propios módulos ESP) son una opción adecuada. A pesar de tener a disponibilidad otros modelos de cámara igualmente viables, como la *OV3660* de 3 Megapíxeles que proporciona resultados similares, se opta por implementar la *OV2640* (Figura 3.5) debido a su reducida resolución, a la sencillez de adquisición y a que su lente posee un ángulo de visión de 68°. Cabe mencionar que existen en su versión acortada o alargada. En este trabajo optamos por la segunda, ya que su flexibilidad ayudará al posterior montaje en la gorra.



Figura 3.5: Cámara *OV2640* en distintas formas y ángulos de visión

Protocolos de comunicación

El sistema principal de este dispositivo tiene múltiples componentes operando a la vez y cumpliendo distintas funciones, por lo que es necesario establecer sistemas de comunicación convenientes para garantizar que los datos se transmitan de la manera más segura, rápida y eficiente posible sin comprometer la información y el pleno funcionamiento del prototipo final.

Como se advirtió previamente, el hardware del sistema se estructura en tres bloques fundamentales:

- Un microcontrolador encargado de capturar y procesar la imagen.
- Un microcontrolador encargado de interconectar el sistema de captura y procesamiento con la interfaz de usuario.
- Un dispositivo externo que aloje la interfaz de usuario y muestre de forma didáctica e intuitiva la información.

Bajo esta arquitectura, se establecen dos canales de comunicación principales. El primero de ellos vincula a ambos microcontroladores y el segundo enlaza al microcontrolador secundario con el dispositivo externo.

Respecto al primer enlace, cabe aclarar que no se transmiten los fotogramas del video, sino únicamente los resultados del procesamiento del detector ocular: las coordenadas espaciales (*X* e *Y*) de la pupila y un valor booleano que indica si el ojo se encuentra abierto o cerrado. Puesto que el sistema debe operar en tiempo real, esta transmisión exige alta velocidad y estricta seguridad para evitar la pérdida de paquetes. Para satisfacer estos requisitos, sumado a factores de practicidad física en el diseño, se determinó inicialmente la implementación del protocolo cableado *SPI* (Serial Peripheral Interface).

El protocolo SPI es un estándar de comunicación de datos en serie normalmente utilizado para comunicar un dispositivo llamado *maestro* con otro llamado *esclavo* (puede haber uno o varios de cada categoría). Esta forma de comunicación es ideal debido a que es:

- Sincrónica: Ambas partes están sincronizadas con una misma señal de clock a la hora de comunicarse, lo que garantiza que ningún dato se pierda en el proceso.
- De baja latencia: A diferencia de otros protocolos, como *UART*, el protocolo SPI es capaz de operar a frecuencias significativamente altas (del orden de los Megahertz) sin perder estabilidad.

Estas dos ventajas satisfacen los requisitos de velocidad y seguridad necesarios para la transmisión de las coordenadas de la pupila. En la figura 3.6 se ilustra este protocolo de comunicación.

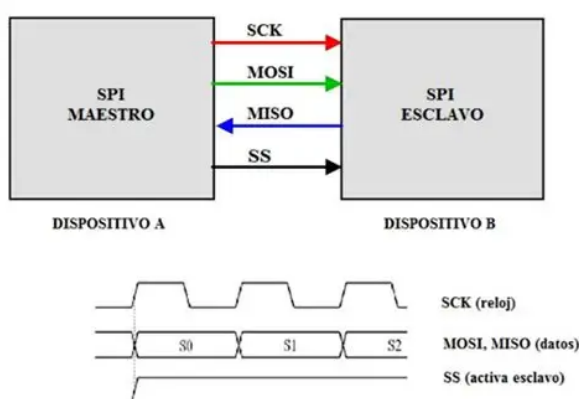


Figura 3.6: Funcionamiento del protocolo SPI

Analizando la comunicación entre el microcontrolador que ejecuta la aplicación web y el dispositivo periférico que muestra la interfaz de usuario, se descarta de entrada un canal de transmisión por cable, ya que el receptor no necesariamente tendrá las mismas características entre una sesión y otra: puede tratarse de una laptop, un celular, una tablet o una computadora de escritorio, y resulta complicado estandarizar un medio físico para todos ellos, además de que se prioriza la comodidad del usuario. Es por eso que se opta por un canal de transmisión inalámbrico, para lo cual se analizaron tres posibilidades detalladas en la tabla 3.1

Tabla 3.1: Sistemas de comunicación inalámbrica

	Bluetooth	Wi-Fi	RadioFrecuencias
Alcance	Bajo (unos pocos cm)	Medio (varios metros)	Muy alto (Kilómetros)
Accesibilidad de hardware	La gran mayoría de dispositivos soporta Bluetooth	Prácticamente todos los dispositivos soportan Wi-Fi y tienen un navegador instalado	Generalmente requieren hardware externo para leer radiofrecuencias
Accesibilidad de Software	Se requiere una app nativa para interpretar los datos	Soporta protocolo TCP-IP	Se requiere una app nativa para interpretar los datos

Por todos los motivos anteriormente evaluados, se opta por utilizar conexión Wi-Fi.

Entornos de desarrollo

La programación del hardware requiere la utilización de un Entorno de Desarrollo Integrado (IDE) para escribir, compilar y cargar el código en la placa de desarrollo. Para este proyecto se optó por el software *Visual Studio Code* junto con la extensión *PlatformIO*, ya que su eficiencia en la gestión de dependencias y librerías facilita el trabajo con una amplia variedad de microcontroladores. Adicionalmente se realizaron pruebas utilizando *Arduino IDE*, que también soporta el hardware de Espressif.

Dado que los microcontroladores de la familia ESP32 son compatibles con las librerías de Arduino, esta alternativa (*Visual Studio + PlatformIO*) permitió una evaluación más ágil del microcontrolador durante las etapas iniciales de desarrollo en comparación con el entorno oficial provisto por Espressif (*ESP-IDF*). Por otra parte, para la etapa de experimentación y entrenamiento de los modelos matemáticos de redes neuronales orientados a la detección ocular, se utilizó el entorno basado en la nube *Google Colab*, el cual permite la ejecución de código Python en notebooks interactivos.



Figura 3.7: Entornos de desarrollo (*Visual Studio Code*, *PlatformIO*, *Arduino IDE* y *Google Colab*)

3.2. El detector ocular

En esta sección se describe el desarrollo del sistema de detección ocular, el cual experimentó múltiples modificaciones e iteraciones hasta llegar al resultado final.

3.2.1. Procesamiento local

En un principio se estableció que el procesamiento de imágenes se realizaría en una de las placas *ESP32-S3* mediante una red neuronal artificial de tipo convolucional. Al ser el propio microcontrolador el encargado de gestionar todas estas tareas de detección es que se habla de **procesamiento local**. Lograr que un microcontrolador ejecute una red neuronal de detección de ese calibre no es tarea sencilla, puesto que la mayoría de estos modelos exigen una alta capacidad de memoria y velocidad que suele exceder a lo que el *ESP32-S3* puede ofrecer. Es por esto que se comenzó a explorar formas de diseñar y entrenar redes neuronales lo más livianas y sencillas posibles. A este paradigma de trabajo que busca la implementación de inteligencia artificial con recursos de software y hardware limitados se lo conoce como **Tiny Machine Learning** (o **TinyML**), y es la filosofía que se aplicó en el desarrollo de este dispositivo.

Edge Impulse

Un primer acercamiento a la disciplina del *Tiny Machine Learning* puede darse con el uso de *Edge Impulse*, una plataforma de desarrollo basada en la nube orientada específicamente al aprendizaje automático con bajos recursos computacionales (Figura 3.8). Fundado en 2019 con el objetivo de facilitar esta tarea, su principal función es trasladar el procesamiento desde servidores en la nube directamente al hardware local. Es una plataforma muy versátil, ya que ofrece compatibilidad con los entornos de desarrollo previamente mencionados y con el framework de Arduino.

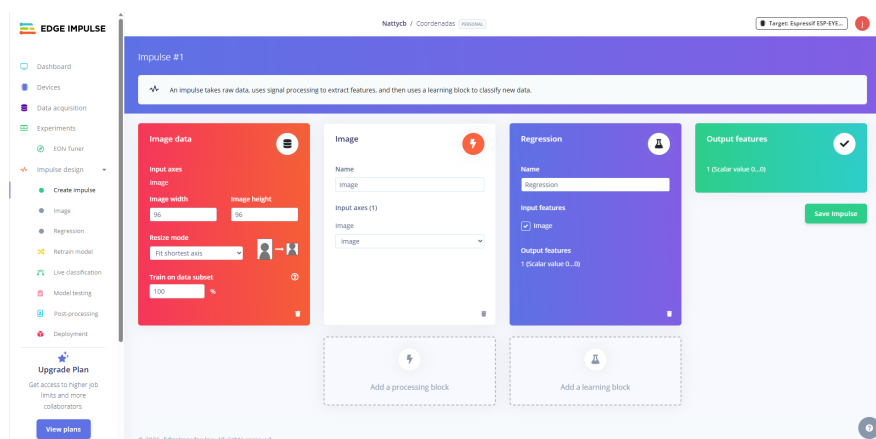


Figura 3.8: Pantalla de creación de modelos de *Edge Impulse*

Su funcionamiento se basa en una estructura de trabajo lineal (conocido como *pipeline* o *Impulse*). Este flujo unifica todas las etapas del desarrollo de un modelo predictivo: comienza con la adquisición y etiquetado de datos estructurados, continúa con bloques de procesamiento digital de señales (para extraer las características más relevantes de imágenes o audio), sigue con el entrenamiento de la red neuronal propiamente dicha y finaliza con el empaquetado del modelo. Todo esto mediante una interfaz de usuario intuitiva.

Con las técnicas de cuantización que realiza y sus métodos de aceleración de hardware, resulta una opción prometedora que optimiza todo el proceso de aprendizaje. Funciona muy bien para tareas sencillas como reconocimiento de objetos o rostros (Figura 3.9).

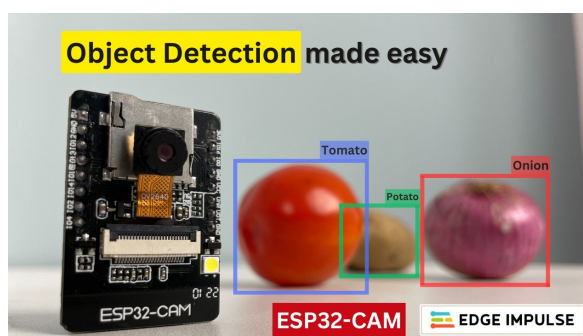


Figura 3.9: Reconocimiento de vegetales con *EdgeImpulse*

Sin embargo, tras varias pruebas realizadas con *datasets* (conjunto de datos de entrada conocidos) sencillos, se determinó que este método no alcanzaba la precisión requerida para la detección continua de coordenadas espaciales a tiempo real, o siquiera para determinar hacia qué dirección se dirigía la mirada (arriba, abajo, izquierda o derecha).

Además, al momento de cargar la red exportada en el procesador *ESP32*, el monitor serial evidenció que cada inferencia demoraba entre 700 y 1000 milisegundos en finalizar, una latencia sumamente indeseable cuando se busca un seguimiento en tiempo real.

Debido a estas imprecisiones y demoras se decidió descartar esta solución.

(Imágenes de la imprecisión o el delay?)

TensorFlow Lite

Una herramienta muy utilizada en Machine Learning es el framework de código abierto *TensorFlow* (*TF*), el cual está diseñado para entrenar y ejecutar modelos de redes neuronales propias o ya existentes. El principal problema radica en que este entorno no está optimizado para ejecutar estas redes en sistemas con hardware limitado como lo es el microcontrolador *ESP32*, y es por eso que Google desarrolló una versión más liviana dedicada a facilitar este tipo de situaciones, llamada *TensorFlow Lite* (*TFLite*), junto con su variante específica para arquitecturas sin sistema operativo: *TensorFlow Lite for Microcontrollers* (*TFLM*).

Como primera instancia se intentó desarrollar un modelo propio con datos de entrada tomados por el *ESP32* y mantener un control absoluto de la optimización y exportación del modelo. Esto último es una enorme ventaja que se tiene respecto al uso de *Edge Impulse*, ya que permite gestionar exactamente cómo se comprime la red para el microcontrolador, evitando las limitaciones y la sobrecarga de código de una plataforma cerrada.

Para facilitar el proceso de creación del modelo, Google ofrece también la herramienta *Keras*, una API de alto nivel que agiliza y simplifica enormemente la programación de la arquitectura neuronal. Es por esto que el entorno de desarrollo seleccionado para esta tarea fue *Google Colab* (Figura 3.10) utilizando el lenguaje Python.

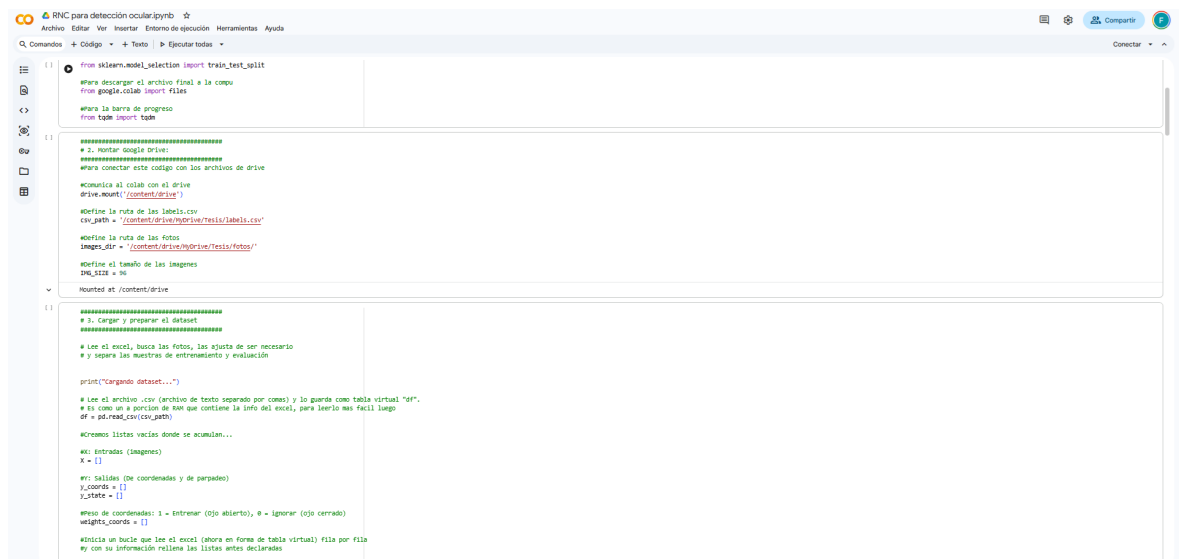


Figura 3.10: Entorno de *Google Colab*

Antes de empezar a codificar es necesario un *dataset*. Para una red de este tipo, normalmente se utilizan cientos de imágenes asociadas a lo que se denominan *etiquetas*. Una etiqueta representa el valor objetivo que el modelo debe predecir y aprender. Para este proyecto, las etiquetas corresponden a números enteros fijos que representan las coordenadas espaciales (*X* e *Y*) del centro de la pupila.

Las imágenes de entrenamiento fueron capturadas utilizando el módulo *ESP32-CAM* en formato de escala de grises (*Grayscale*) y con una resolución de 96x96 píxeles. Esta configuración se seleccionó con el propósito de minimizar el consumo de recursos computacionales sin comprometer los detalles necesarios en la imagen. Para garantizar cantidad y variabilidad en los datos, se tomó un conjunto total de 1315 capturas obtenidas de cinco personas distintas, contemplando distintas condiciones de iluminación y variaciones físicas como la presencia y ausencia de anteojos.

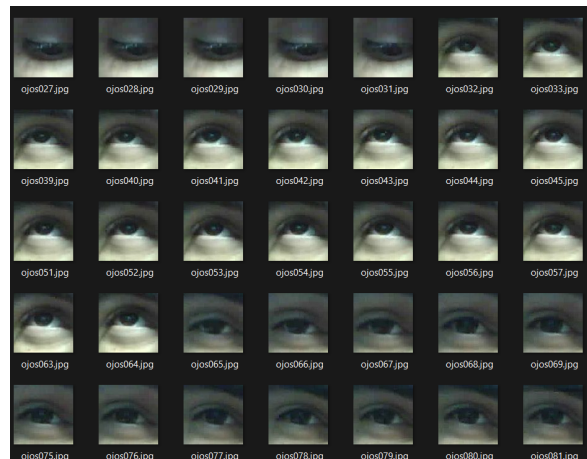


Figura 3.11: Una muestra de las imágenes utilizadas para el dataset

En paralelo, las coordenadas correspondientes a cada imagen (expresadas en píxeles dentro de una imagen 96x96), se anotaron en una hoja de cálculo, estructurando así los datos para su posterior utilización en el entrenamiento.

labels					
Archivo Editar Ver Insertar Formato Datos Herramientas G					
Menús					
K7					
	A	B	C	D	E
1	filename	x	y		
2	ojos0001	41	47		
3	ojos0002	41	47		
4	ojos0003	41	47		
5	ojos0004	41	47		
6	ojos0005	41	47		
7	ojos0006	41	47		
8	ojos0007	41	47		
9	ojos0008	43	46		
10	ojos0009	24	48		
11	ojos0010	18	48		
12	ojos0011	18	48		
13	ojos0012	18	48		
14	ojos0013	18	48		
15	ojos0014	18	48		
16	ojos0015	50	28		
17	ojos0016	50	28		
18	ojos0017	50	28		
19	ojos0018	50	28		
20	ojos0019	50	28		
21	ojos0020	50	28		

Figura 3.12: Planilla con etiquetado de imágenes

A continuación se listan y explican brevemente las etapas de la creación de un modelo de Red Neuronal Convolutiva (CNN) en *Google Colab* con *Python/Keras*:

1. Preparación: Se comienza importando en el código las herramientas necesarias para construir la CNN. Entre ellas *Tensorflow*, *Pandas*, *Numpy*, *OpenCV*, entre otros. Se monta la herramienta de *Google Drive*, para que el entorno pueda acceder a las imágenes y etiquetas almacenadas allí.
2. Formateo del dataset: Se extraen de *Google Drive* las 1315 imágenes, se las recorta a 96x96 de ser necesario y se leen y almacenan las etiquetas en arreglos.
3. Creación de la red: Se crea el modelo indicando el formato de sus entradas, sus capas internas y su función de activación. Al final se compila el modelo indicando cómo trabajará con los errores.
4. Separación de datos: Aquí se aplica una técnica para entrenar y evaluar la red al mismo tiempo. Se separan las imágenes del dataset en dos grupos: el grupo de entrenamiento (un 80 % de las imágenes) y el grupo de evaluación (el 20 % restante). Esta separación es aleatoria y sirve para comprobar el resultado de la construcción del modelo.
5. Entrenamiento: Se realiza el entrenamiento en sí mismo eligiendo el número de *épocas* (veces que el modelo analiza el grupo de imágenes) y aplicando *callbacks* (funciones que detienen el entrenamiento cuando se alcanza la precisión deseada), para evitar que la red memorice fotos en lugar de aprender de ellas.
6. Evaluaciones: Se pone a prueba la red insertándole la porción de datos que nunca vió durante su entrenamiento. Se muestran los resultados y análisis de distintos parámetros estadísticos como el porcentaje de aciertos, los errores, la muestra con peor desviación, etc.
7. Exportación: Se exporta el modelo a un archivo en formato *.tflite* para ser utilizado en el entorno de *Visual Studio Code*.

Tras iterar sobre la arquitectura de la red y ajustar sus hiperparámetros, tales como la cantidad de capas y los filtros de procesamiento, para maximizar la precisión (Figura 3.13), se procedió a exportar el modelo y compilarlo en el entorno de C++ para evaluar su rendimiento en vivo.



Figura 3.13: Resultados del modelo desarrollado en *Google Colab*

Si bien este modelo demostró ser significativamente más rápido, eficiente y controlable que la opción generada mediante *Edge Impulse*, las pruebas empíricas en tiempo real mostraron que no cumplía con los requisitos de precisión para ser utilizado en el dispositivo final. Al operar el sistema, el modelo presentaba una alta inestabilidad en las predicciones: el cursor de la interfaz sufría de oscilaciones pequeñas y constantes (*jittering*) y era incapaz de mantener una posición firme o realizar un seguimiento fluido de la mirada.

(Imágenes de las fallas?)

Al intentar reducir esa inestabilidad, se encontró un límite de hardware insalvable: aumentar la complejidad y profundidad de la red neuronal para estabilizar la predicción disparaba la latencia de procesamiento a niveles inaceptables, mientras que simplificar la arquitectura para recuperar velocidad degradaba la exactitud de las coordenadas.

(Imágenes del monitor serial con los delays?)

A partir de este compromiso insuperable entre latencia y precisión, se concluye que el procesamiento neuronal de imágenes de forma puramente local dentro de las limitaciones de memoria del microcontrolador no resulta viable para las exigencias de este proyecto. Esta limitación motiva el descarte definitivo en el uso de redes neuronales en hardware de bajos recursos. Sin embargo, para mantener el enfoque de este trabajo, no se sustituirá el hardware base, sino que se buscará otro método con un procesamiento más sencillo.

Detección por umbral de oscuridad y brillantez

Al no tener éxito en el procesamiento de imágenes mediante el uso de redes neuronales convolucionales, se optó por un método de visión computacional clásica. Este consiste en capturar constantemente una imagen en escala de grises y analizar en tiempo real sus patrones de oscuridad (intensidad lumínica).

Cuando se toma una imagen del ojo de un individuo, en la inmensa mayoría de los casos el punto más oscuro resulta ser la pupila (Figura 3.14), puesto que el resto de la imagen está compuesto por porciones irregulares de distintos tonos grises más claros. Este método aprovecha esa oscuridad extrema para señalar con un cursor la ubicación exacta de la pupila, y por lo tanto inferir la dirección de la mirada.

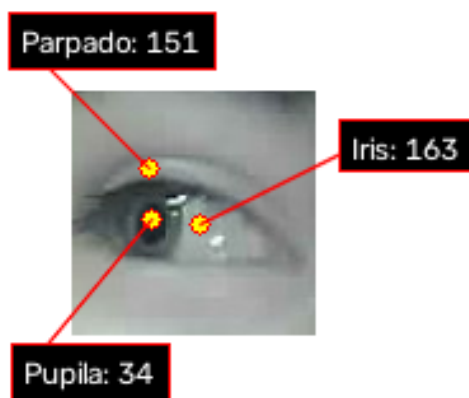


Figura 3.14: Distintas partes del ojo y su nivel de oscuridad

Para entender este método se debe aclarar previamente la manera en que el microcontrolador procesa una imagen. Al capturar un fotograma, el *ESP32* lo almacena en un *framebuffer*, es decir, un bloque de memoria reservado estructurado como una matriz bidimensional. Si se trabaja en escala de grises, cada elemento de esta matriz contiene un número entero comprendido entre 0 y 255. Este valor representa la intensidad lumínica del píxel en esa posición espacial específica (donde 0 equivale a oscuridad absoluta y 255 a saturación blanca).

El algoritmo procesa la imagen realizando un barrido inicial en el que revisa cada píxel para identificar el valor de intensidad mínimo (el punto más oscuro del fotograma). Para evitar que la detección dependa de un único píxel, lo cual arriesgaría la estabilidad, se establece un umbral de oscuridad dinámico. Esto significa que el algoritmo no aísla solamente el píxel más oscuro,

sino a toda una región de píxeles cuyos valores de intensidad se encuentren por debajo de dicho umbral.

Una vez seleccionada la región de interés (*ROI*) que teóricamente abarca a la pupila, el programa ejecuta un segundo barrido para calcular sus coordenadas centrales mediante la aplicación de un centroide ponderado. A diferencia de un promedio simple, donde se suman todas las coordenadas y se dividen por la cantidad de píxeles detectados, el centroide ponderado asigna un factor de peso a cada coordenada en función de su nivel de oscuridad. Mientras más oscuro sea un píxel, mayor será su peso matemático en la ecuación final. Esta ponderación resulta fundamental para la estabilidad del sistema, ya que otorga una influencia dominante al núcleo de la pupila (altamente oscura) y reduce considerablemente las desviaciones causadas por sombras periféricas o pestañas que hayan ingresado dentro del umbral de detección (grises más claros).

(Imagen del cálculo con centroide en el código?)

Este algoritmo, mucho más liviano que una red neuronal convolucional, redujo considerablemente el tiempo de procesamiento y aumentó la eficiencia de recursos de memoria en el *ESP32*. Sin embargo, presentaba un problema importante: la iluminación ambiental. Al evaluar este método se descubrió que su precisión dependía exclusivamente de la luminosidad de la imagen. Un ambiente con mucha luz genera sombras pronunciadas que el algoritmo puede confundir con las pupilas. Por el contrario, en un ambiente con poca luz, el nivel de oscuridad de la pupila y de otras áreas (como pestañas o cejas) se vuelve demasiado similar. A pesar de que el umbral de oscuridad puede ajustarse mediante código, resulta poco práctico si se considera que la luz ambiental es una variable incontrolable y sumamente dinámica, motivo por el cual este algoritmo se descarta como una solución óptima para la detección ocular definitiva.

A pesar de que la iluminación ambiental es una variable incontrolable, lo que sí se puede controlar es la iluminación de la imagen captada por el *ESP32*. Si hubiese una fuente de luz junto a la cámara, esta variable sería siempre constante y se evitaría confusiones con sombras o pestañas. Sin embargo, alumbrar el ojo del usuario con luz visible permanentemente sería molesto y un perjuicio para su visión, por lo que se tuvo que plantear un método de iluminación que solo la cámara del microcontrolador pudiera detectar sin dañar la vista: la luz infrarroja (IR). Este tipo de iluminación tiene longitudes de onda mayores a las de la luz visible, por lo que es indetectable por el ojo humano, mientras que los sensores de las cámaras digitales sí son capaces de captarla. Es por eso que colocar un diodo LED emisor de luz infrarroja junto a la cámara parecía ser la solución perfecta para este problema, pero había otro impedimento físico a superar: el efecto del brillo corneal.

Al ser húmeda y curva, la superficie de la córnea del ojo actúa como un espejo esférico perfecto para estas ondas infrarrojas. La luz rebota en la superficie del ojo generando un punto blanco muy intenso y concentrado (denominado *Glint*). Esto ocasionó errores severos en el algoritmo anterior de detección de oscuridad, por lo que se decidió modificarlo para utilizar este fenómeno físico a favor: en lugar de detectar los píxeles más oscuros, se invirtió la lógica para detectar los píxeles más claros mediante un umbral de brillantez.

(Imagen del glint?)

Esta solución tampoco resultó viable. Al realizar las pruebas de seguimiento, se comprobó que el glint no se desplaza de la misma forma que la pupila: su posición relativa depende estrictamente de la geometría del ojo frente a la fuente de luz.

(Imagen del glint desplazado?)

De este modo, tras exhaustivos análisis, pruebas y cambios de lógica, se demostró que estos algoritmos de detección de oscuridad y brillantez no alcanzaban la precisión requerida para el

correcto funcionamiento del dispositivo. En consecuencia, se concluye que el procesamiento de imagen directamente en el microcontrolador no cumple con las expectativas impuestas al inicio de este trabajo, por lo que queda descartado definitivamente.

3.2.2. Procesamiento remoto

Tras las limitaciones técnicas comprobadas en la aplicación de métodos de detección ocular utilizando los recursos de un microcontrolador, se optó por prescindir del procesamiento local en el hardware base y derivar esta tarea a un dispositivo externo. Por este motivo, los métodos expuestos a continuación se engloban en la categoría de *procesamiento remoto*. En este sentido, el microcontrolador se encarga exclusivamente de capturar las imágenes y alojar la aplicación web, mientras que el procesamiento matemático de los fotogramas es ejecutado por el dispositivo cliente. Debido a la reducción considerable de la exigencia al microcontrolador capturador (pues su única tarea es capturar fotogramas y transmitirlos), se optó por ahorrar más recursos utilizando un *ESP32-CAM*, una versión menos potente y más económica que el *ESP32-S3*.

Redes neuronales pre-entrenadas

Al no tener la limitación de la velocidad de procesamiento del microcontrolador, se puede optar por utilizar una red neuronal pre-fabricada más potente para la detección de coordenadas. Dos alternativas consideradas fueron *MediaPipe* (un conjunto de librerías provisto por Google) y *WebGazer* (un modelo desarrollado en el ámbito académico). Ambas son herramientas basadas en tecnologías web que solucionarían todos los problemas de iluminación ambiental descritos con anterioridad, pero fueron rápidamente descartadas debido a dos inconvenientes.

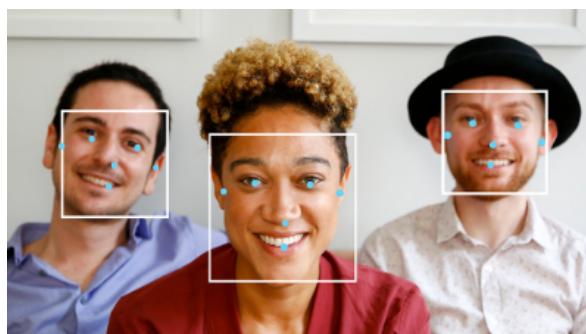


Figura 3.15: Detección de rostro con *MediaPipe*

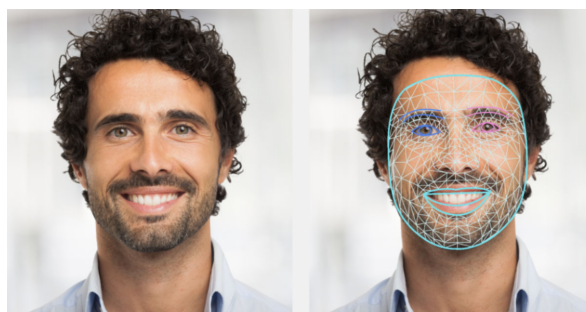
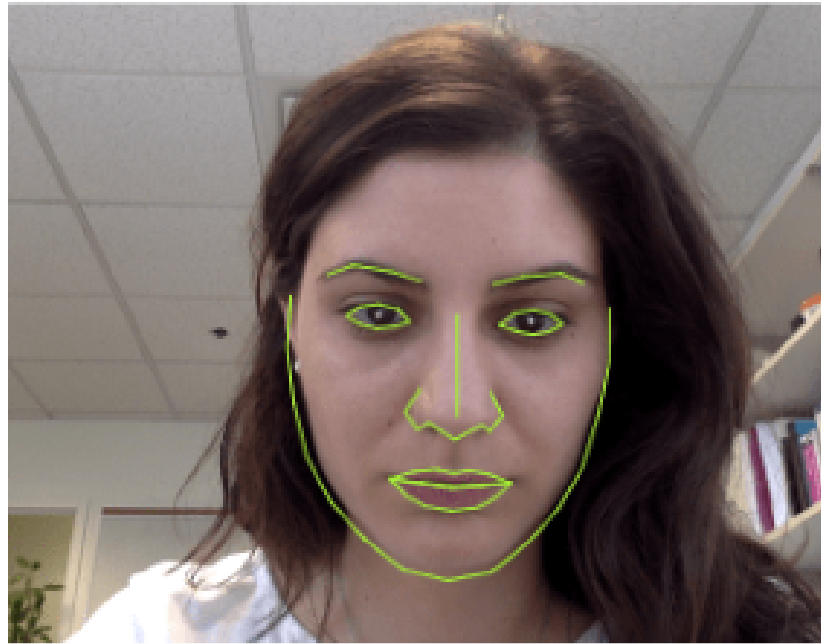


Figura 3.16: Facciones del rostro en *MediaPipe*

Figura 3.17: Facciones del rostro en *WebGazer*

Por un lado, ambos son detectores de expresiones faciales y no están diseñados para seguir específicamente el movimiento de la pupila. Si bien los algoritmos se pueden adaptar, requieren una vista completa del rostro para poder anclar sus referencias. Esto implica perder precisión en las coordenadas exactas del ojo (al tener que medir variaciones minúsculas en píxeles) e invalida el diseño físico planeado originalmente para este proyecto (una cámara que capte únicamente la región ocular).

El otro inconveniente es que estas redes son demandantes en términos del hardware de captura necesario para su ejecución. Utilizar la cámara de baja resolución del *ESP32* de forma focalizada no es la condición ideal para que estos modelos operen con normalidad. Se podría deducir que esto se soluciona derivando la tarea de captura de imagen a las cámaras integradas de los dispositivos externos (PC, celular o tablet), pero esto comprometería una característica vital del dispositivo final: la versatilidad. Cada dispositivo cliente posee una cámara con propiedades distintas, variando drásticamente su resolución, ubicación, distancia focal e inclinación. Todas estas son variables incontrolables que afectarían la precisión del seguimiento espacial, algo que se evita por completo al utilizar un dispositivo universal que mantiene siempre su propia cámara en una posición fija y calibrada respecto al ojo. Por todos estos motivos empíricos y de diseño, se decide prescindir definitivamente de las redes neuronales como métodos de detección espacial.

OpenCV con círculos de Hough

La librería *OpenCV* (Open Source Computer Vision Library) es una biblioteca de software de código abierto especializada en visión computacional y procesamiento de imágenes. Su principal ventaja es que está altamente optimizada para aplicaciones en tiempo real. En lugar de tener que escribir desde cero las complejas operaciones matemáticas para leer matrices de píxeles, detectar bordes o filtrar ruido, *OpenCV* ofrece herramientas y funciones prefabricadas que ejecutan estos cálculos aprovechando al máximo la arquitectura del procesador del dispositivo cliente. Por todo lo anterior, además de contar con mayor versatilidad que las redes neuronales anteriormente expuestas, *OpenCV* se perfiló como una alternativa sumamente viable para utilizar en este trabajo.

La principal desventaja de este método radica en el hecho de que desarrollar el detector requiere la creación de una aplicación nativa en Python que cada dispositivo cliente debe instalar y ejecutar por su cuenta. Evaluando ambos aspectos, se optó por aceptar esta dependencia de software en pos de obtener un detector ocular funcional y robusto.

Para aprovechar al máximo las capacidades de *OpenCV*, se decidió complementarlo con un algoritmo matemático que se conoce como la *Transformada de Hough*, un método clásico de extracción de características utilizado en el análisis de imágenes digitales. Su variante para círculos (conocida como *HoughCircles* en *OpenCV*) funciona analizando los bordes detectados en un fotograma mediante un sistema matemático de *votación*. El algoritmo busca conjuntos de píxeles que compartan una curvatura y se encuentren a una distancia constante (el radio) de un posible punto central. Si suficientes píxeles *votan* por el mismo centro, el algoritmo confirma la existencia matemática de un círculo en esa región. En este contexto, el círculo vendría a ser el iris del ojo, y para determinar las coordenadas exactas de la pupila simplemente se debe calcular la ubicación del centro de dicho círculo.

Lo primero se debe hacer es transferir la imagen desde el microcontrolador capturador hacia el dispositivo cliente (recordemos que este método entra en la categoría de *procesamiento remoto*), para lo cual se opta por utilizar una conexión Wi-Fi mediante protocolo *UDP*. Se descarta la transmisión SPI puesto que no es adecuada para manejar el alto ancho de banda que requiere la transmisión fluida de imágenes. Una vez logrado esto (proceso que se detalla posteriormente en este informe), empieza el trabajo del procesador del dispositivo cliente.

Ya recibido el fotograma, *OpenCV* realiza un proceso de filtrado espacial (preprocesamiento) para limpiar la imagen de ruido digital y prepararla para el análisis. A continuación se le aplica la función de círculos de Hough y, en el momento en que se detecta el contorno del iris se grafican los resultados sobre la imagen a modo de depuración visual: un trazo verde delimitando la circunferencia detectada con un punto rojo en su centro que representa las coordenadas de la pupila.

(Imagen de este método funcionando?)

Aquí surge un problema importante. La función de círculos de Hough está matemáticamente diseñada para detectar circunferencias perfectas, y el iris humano no siempre se presenta como tal frente a la cámara: frecuentemente, los párpados ocuyen una parte superior o inferior, alterando así su geometría circular. Esto implicaría que el usuario debería forzar permanentemente sus músculos oculares para mantener los ojos abiertos de forma antinatural y garantizar el correcto funcionamiento del sistema, lo cual resulta poco práctico y sumamente indeseable.

(Imagen con párpados semicerrados?)

Una posible solución consiste en disminuir la exigencia de *OpenCV* para que no detecte únicamente círculos perfectos, sino que tenga cierta tolerancia ante las deformaciones (permitiendo la detección de arcos incompletos o formas elípticas). Esto, sin embargo, genera un problema secundario grave, puesto que el algoritmo se vuelve demasiado permisivo y comienza a registrar falsos positivos al detectar curvas en párpados, cejas, arrugas y otras facciones del rostro.

(Imagen de falsos positivos?)

Ambos problemas presentan un compromiso técnico recíproco, debido a que si se intenta reducir uno, el otro empeora. Luego de sucesivas pruebas utilizando herramientas complementarias como umbrales de radio y detectores de brillantez/oscuridad apoyados por el LED infrarrojo, se concluye que la precisión alcanzada no es la adecuada para un seguidor ocular en tiempo real, por lo que esta solución queda definitivamente descartada.

De esta forma, tras varios intentos insatisfactorios, se toma una decisión radical en el desarrollo de este trabajo: dejar atrás la detección de coordenadas espaciales específicas y pasar a identificar patrones de orientación de la mirada.

Detección de similitud estadística (Template Matching)

Este método busca simplificar el sistema de detección ocular con el fin de disminuir la carga de procesamiento, a la vez que elimina toda posible causa de errores tratada con anterioridad. El objetivo es lograr un algoritmo independiente de la luz ambiental, de la resolución de la cámara y de las variaciones en las formas geométricas biológicas del ojo.

Para evitar, a su vez, la dependencia de aplicaciones nativas de Python, se optó por implementar este algoritmo directamente en una aplicación web. Esta interfaz se aloja en la memoria del propio *ESP32* servidor, y es transmitida al dispositivo cliente externo. Esto implica que el usuario simplemente debe conectarse a la red Wi-Fi generada por el microcontrolador y abrir su navegador de preferencia para poder utilizar el sistema. Los detalles del desarrollo de esta aplicación se exponen posteriormente en este informe.

(Diagrama de bloques del nuevo sistema?)

Este sistema de detección utilizará el método conocido como *Template Matching* (ya introducido en secciones anteriores) y aprovechará su fácil implementación mediante OpenCV para realizar un detector sencillo de dirección de mirada.

Este método requiere, como paso inicial, un proceso de calibración que otorgue las plantillas a utilizar. Para ello se captura un número definido de imágenes que representarán todas las acciones posibles dentro de la interfaz. Para garantizar la máxima estabilidad y mitigar posibles confusiones en el algoritmo, se optó por limitar estas acciones a seis:

- Mirada hacia arriba a la izquierda (Seleccionar opción 1)
- Mirada hacia arriba a la derecha (Seleccionar opción 2)
- Mirada hacia abajo a la izquierda (Seleccionar opción 3)
- Mirada hacia abajo a la derecha (Seleccionar opción 4)
- Mirada hacia el centro de la pantalla (Zona neutra, no realiza ninguna acción)
- Parpadeo (Confirmar opción elegida)

(Imagen de los seis moldes?)

Una vez capturadas estas seis imágenes (con resolución 96x96 en escala de grises), se almacenan temporalmente en la memoria del navegador para tenerlas a disposición durante la posterior ejecución. Con el sistema en marcha, el *ESP32* captura el ojo en tiempo real con una resolución nativa de 320x240 píxeles en formato de color estándar y envía los fotogramas al navegador web mediante la red local. Es allí, aprovechando los recursos del dispositivo cliente, donde se produce el procesamiento comparativo de las imágenes.

(Imágenes del stream?)

Se comienza con un previo ajuste de la imagen a procesar, esto es, recortar la imagen a un tamaño de 128x128 y convertir su formato a escala de grises para reducir lo máximo posible la carga del procesamiento posterior. Con este mismo fin de optimización, se limita la tasa de procesamiento a 15 fotogramas por segundo (FPS), una frecuencia empíricamente suficiente para garantizar una interacción fluida en tiempo real sin saturar el enlace de transmisión. A

continuación el algoritmo realiza el barrido que compara cada una de las imágenes de la plantilla con la imagen a analizar.

Durante cada comparación individual se emplea un método estadístico denominado *Coefficiente de correlación normalizado con media cero*. Esta herramienta deriva del clásico *Coefficiente de correlación de Pearson*, pero adaptado para operar sobre matrices bidimensionales (que en este caso representan a las imágenes). La fórmula matemática aplicada se define de la siguiente manera:

$$R(x, y) = \frac{\sum_{x', y'} (T(x', y') - \bar{T}) \cdot (I(x + x', y + y') - \bar{I})}{\sqrt{\sum_{x', y'} (T(x', y') - \bar{T})^2 \cdot \sum_{x', y'} (I(x + x', y + y') - \bar{I})^2}}$$

Figura 3.18: Coeficiente de correlación normalizado con media cero.

Donde T (*Template*) es la matriz de la imagen de referencia tomada en la calibración, e I (*Image*) es la matriz de la imagen del fotograma actual.

Como se puede observar en el numerador de la ecuación, a cada píxel se le resta el valor promedio matemático de su respectiva matriz (motivo por el cual recibe el nombre de *media cero*). Esto es clave para el algoritmo de detección, ya que no evalúa la intensidad lumínica absoluta de un píxel, sino qué tan diferente es respecto a su entorno más cercano. Así, por ejemplo, si el rostro del usuario se ilumina súbitamente por una fuente externa, todos los píxeles de la imagen aumentarán su valor, pero la dispersión relativa entre ellos permanecerá constante. Esto le proporciona al sistema una total inmunidad frente a las variaciones incontrolables de la iluminación ambiental. Como se mencionó anteriormente, la imagen de referencia (*Template*) tiene una resolución de 96x96 píxeles, mientras que el fotograma a analizar posee una resolución mayor, de 128x128 píxeles, a pesar de que en ambos casos la imagen del ojo es del mismo tamaño. El motivo de esta diferencia es lograr que la función de correlación no se aplique una sola vez, sino de forma iterativa, permitiendo que el molde realice un barrido completo sobre la imagen en vivo. Esto significa que el algoritmo comienza superponiendo las coordenadas de origen ($X=0$, $Y=0$) de ambas matrices y calculando la fórmula estadística, luego desplaza el molde un píxel hacia la derecha ($X=1$, $Y=0$), vuelve a aplicar la ecuación y repite este ciclo hasta cubrir toda el área del fotograma. Al finalizar este escaneo, se obtiene una nueva matriz bidimensional que contiene todos los coeficientes resultantes de estas sucesivas aplicaciones.

(Imagen ilustrativa del proceso de barrido)

Este minucioso proceso de análisis se justifica por los inevitables errores de posicionamiento relativos entre el ojo y la cámara. Al utilizar una fórmula matemática tan estricta, si se evaluara una única posición estática, cualquier mínima perturbación del dispositivo (incluso de un milímetro), desalinearía las matrices y provocaría el fracaso inmediato del cálculo. Por lo tanto, realizar un barrido completo otorga un margen geométrico para compensar cualquier desplazamiento involuntario. Con esta técnica se logra una ventaja ergonómica fundamental para el proyecto: la inmunidad del sistema ante los micro-movimientos físicos de la cabeza del usuario. De esta matriz resultante con todos los valores de correlación, se extrae el coeficiente más alto (correspondiente al ciclo de comparación en el que la plantilla se alineó con mayor precisión sobre el fotograma capturado). Este proceso se repite con los cinco moldes restantes y, de esos seis valores máximos, se determina cuál fue el mayor, identificando así a qué plantilla corresponde esa coincidencia y, por lo tanto, a cuál acción del ojo corresponde.

(Imagen del log indicando el mayor parecido)

Una vez identificada la acción de mayor probabilidad, el sistema aplica la siguiente lógica de control:

- **Mirada hacia alguna opción (esquinas):** La interfaz resalta visualmente esa opción y la guarda temporalmente en memoria como la última acción válida.
- **Mirada hacia el centro (zona neutra):** El sistema mantiene iluminada la última opción válida seleccionada, permitiendo al usuario descansar la vista sin alterar la pantalla.
- **Parpadeo (ojo cerrado):** El algoritmo actúa como un disparador de confirmación, ejecutando la decisión final si existía una opción previamente seleccionada.

Si bien este método de detección resultó mucho más robusto y funcional que todos los anteriores, en la práctica presenta algunos errores. Por un lado, aunque el algoritmo estadístico es inmune a las variaciones lineales de luz ambiental, las sombras proyectadas por la inclinación de una fuente de luz externa entorpecen el procesamiento. Esto justificó la decisión de reutilizar el diodo LED infrarrojo (IR) en el diseño, garantizando una iluminación frontal uniforme y la consecuente ausencia de estas sombras. El otro inconveniente detectado fue la susceptibilidad a los falsos positivos. En ocasiones, el algoritmo resultaba demasiado permisivo y confundía las orientaciones del ojo. Para mitigar este efecto se introdujo por código un *umbral de similitud*. Esto implica que toda coincidencia probabilística inferior a este valor será automáticamente ignorada por el sistema. Por defecto, este umbral se estableció empíricamente en un 80 % (aunque el usuario puede ajustarlo en todo momento desde la propia interfaz web), solucionando el problema con gran eficacia.

(Imagen del slider)

Teniendo en consideración todo lo estudiado hasta el momento y habiendo superado múltiples ensayos empíricos, se concluye que este método de seguimiento basado en la similaridad de patrones es el más efectivo, estable y eficiente. Por lo tanto, esta es la solución implementada en el dispositivo final.

3.3. Intercomunicación de hardware

En esta sección se explora la manera en la que se comunican todos los dispositivos que actúan en el sistema final. Se hablará de los datos que se transmiten a través de dichos canales de comunicación y la forma en la que se transmiten.

3.3.1. Clientes y servidores

Como se aclaró en una sección previa, la arquitectura principal utilizada en este proyecto se denomina *Cliente-Servidor*, donde los clientes realizan pedidos a través de los enlaces de comunicación (en este caso particular, comunicación Wi-Fi) y los servidores responden con datos. Quien adopta el rol de servidor central es el segundo *ESP32-S3*, que no realiza la tarea de captura de fotogramas. Sus funciones principales son:

- **Gestión de base de datos:** Alojar y administrar la información en una memoria microSD, accediendo a ella cada vez que un cliente solicita datos de usuarios, métricas o cuestionarios.
- **Alojamiento web (Web Server):** Contener los archivos estáticos de la aplicación web en su memoria mediante un sistema de archivos interno (como *SPIFFS* o *LittleFS*), para servirlos a los navegadores de los dispositivos que se conecten.

- **Enrutamiento de video:** Actuar como puente o punto de interconexión para permitir la transmisión continua de fotogramas desde el módulo capturador (*ESP32-CAM*) hacia el navegador del usuario.
- **Lógica de negocio (Backend):** Administrar permisos, validar sesiones, procesar peticiones HTTP, manejar la lectura/escritura en la base de datos y coordinar toda la lógica interna del sistema.

Por otro lado, los clientes son aquellos nodos de la red que carecen de las funciones de ruteo y almacenamiento central, limitándose a solicitar recursos al servidor para complementar el sistema. En este contexto existen dos tipos de clientes bien diferenciados.

Uno de ellos es el módulo *ESP32-CAM*, el cual actúa como cliente de transmisión. Su función es solicitar un enlace en la red para enviar el flujo constante de fotogramas capturados. Los demás clientes son los dispositivos de usuario finales (el celular del alumno, la computadora del profesor o la tablet de un tutor). Su función de hardware es simplemente proveer un navegador capaz de descargar y ejecutar la aplicación web (*frontend*), donde se renderiza la interfaz visual. Para lograr esto y mantener el sistema operativo, estos dispositivos solicitan datos continuamente al servidor tales como el video en tiempo real, el estado de las sesiones, los cuestionarios activos o cualquier otra variable alojada en la base de datos necesaria para la ejecución del programa.

(Diagrama más específico del sistema)

3.3.2. Servidor Wi-Fi local (Naty)

3.3.3. Transmisión de datos (Naty)

3.4. Base de datos (Naty)

3.5. Aplicación web

En esta sección se describen todas las funcionalidades de la aplicación web (*frontend*) y el gestor encargado de asegurar su correcto funcionamiento (*backend*)

3.5.1. Frontend

El *frontend* es la parte del código que se muestra ante el usuario. Su función es ser el intermediario entre el *backend* (quien gestiona los datos y los permisos), y el usuario. En este trabajo se optó por desarrollarlo en *React*, una librería de *JavaScript* de código abierto, creada y mantenida por *META*, diseñada específicamente para construir interfaces de usuario (UI) dinámicas e interactivas para aplicaciones web. Es una herramienta muy versátil, ya que se puede utilizar en el entorno de *Visual Studio Code* y trabaja con arquitectura basada en componentes (CBA), lo cual vuelve al código más modular, ordenado y sencillo de depurar. La estructura del programa se divide por páginas o pantallas. Cada pantalla cumple un conjunto limitado de funciones que conforman un subsistema común. Para funcionar requieren de dos archivos: un archivo *.jsx* que declara y enlaza cada entidad con la función que deben cumplir (botones, campos de texto, títulos, entre otros) y un archivo *.css* que describe el aspecto de cada una de estas entidades (el color de los botones, el tamaño del texto que contengan, la extensión de los campos de texto y otros). Las pantallas se podrían dividir en tres categorías:

Pantallas de acceso universal

Son aquellas interfaces a las que cualquier usuario puede acceder independientemente de su rol, permisos o jerarquía. En esta categoría se agrupan cuatro vistas principales: *Home*, *Login*,

Invitado y Alumno. La pantalla *Home* es la interfaz de bienvenida que el usuario visualiza instantáneamente al ingresar a la dirección IP del servidor local. Cuenta con un diseño minimalista que incluye un texto de presentación y dos botones de navegación para indicar qué tipo de usuario está ingresando. Si se presiona el botón destinado al estudiante, el sistema redirige a la pantalla *Alumno*; caso contrario, el botón de usuario externo deriva el flujo hacia la pantalla *Login*.

(Imagen Home)

La pantalla *Login*, como su nombre indica, es aquella que los usuarios que no son alumnos utilizan para identificarse antes de comenzar su sesión con el dispositivo. Los usuarios con jerarquía (profesores y tutores) deben ingresar sus credenciales para acceder al núcleo del sistema. Además de los campos de texto donde se ingresan estos datos, esta vista incorpora un botón para registrar un usuario nuevo, el cual despliega un formulario que solicita datos personales (nombre, apellido, contacto), el rol deseado, la *referencia* (Materia en el caso de los profesores y vínculo en el caso de tutores) y una palabra clave de administrador. Esta clave maestra es un requisito de seguridad indispensable para autorizar la creación de perfiles aptos para utilizar el dispositivo con fines académicos.

(Imagen Login y otra con registro)

Otro botón importante es el de *Recuperar contraseña*, que permite a cualquier usuario restablecer su contraseña en caso de haberla olvidado. Naturalmente, esta acción requiere de la palabra clave maestra.

(Imagen de recuperar contraseña)

Finalmente, la interfaz de inicio de sesión ofrece un botón para ingresar directamente como invitado, ideado para aquellas personas del entorno del estudiante (amigos o familiares) que deseen establecer una comunicación informal sin necesidad de poseer un perfil registrado en la base de datos. Al seleccionar esta última opción, el usuario es redirigido a la pantalla *Invitado*. Esta vista presenta un formulario simplificado de opción múltiple diseñado para estructurar preguntas rápidas. Posee un campo para la pregunta, cuatro para las posibles respuestas, botones para eliminar o agregar opciones (hasta un máximo de cuatro) y un botón para enviar la pregunta al estudiante.

(Imagen de pantalla invitado)

Por otro lado se encuentra la pantalla *Alumno*. Al tratarse del usuario principal y prioritario del sistema, su ingreso no requiere autenticación. Sus funciones exclusivas son calibrar el detector y responder las evaluaciones que lleguen desde otros dispositivos. Al ingresar a esta vista, el usuario observa un panel indicador del estado del sistema y dos botones de acción principal (calibración y examen). Dichos botones permanecen bloqueados por seguridad hasta que el sistema esté en condiciones de operar. Antes de permitir cualquier acción, el frontend le solicita al servidor la descarga del archivo *opencv.js*. Esta librería matemática actuará como el motor de la detección visual, pero adaptada exclusivamente para el método de *Template Matching* (dejando atrás el enfoque previo de los círculos de Hough).

(Imágenes de panel alumno bloqueado y desbloqueado)

Una vez que el indicador confirma que la librería *OpenCV* fue leída desde la tarjeta microSD, transferida y compilada correctamente por el navegador, los botones de acción se habilitan. El primero de ellos dirige a la interfaz de calibración, la cual está compuesta por seis cuadros de previsualización vacíos, un recuadro de mayor tamaño donde se proyecta el flujo de video en vivo y los comandos para iniciar o abortar el proceso. Si el sistema detecta una calibración almacenada previamente, advierte al usuario para evitar posibles redundancias.

(Imagen pantalla de calibración)

Al pulsar el botón de inicio, comienza una secuencia guiada para capturar los fotogramas de referencia: un indicador visual le señala al usuario hacia qué cuadrante debe dirigir la mirada y, tras una cuenta regresiva de tres segundos, el algoritmo captura la imagen. Este procedimiento se repite para todas las direcciones y para el parpadeo final. Una vez concluida la secuencia, los patrones capturados se exhiben en los cuadros de previsualización, otorgando la opción de guardar dicha calibración o reiniciar el proceso.

(Imagen de uno de los pasos de la secuencia)

El segundo botón del panel principal (*Iniciar cuestionario*) redirige a la interfaz operativa de evaluación. Esta pantalla presenta el flujo de video en la esquina superior derecha junto a un control deslizante (*slider*) que permite ajustar dinámicamente el umbral de sensibilidad del algoritmo. En este estado, el cliente aguarda pasivamente hasta que un invitado envíe una consulta o un profesor inicie formalmente un cuestionario. Al recibir los datos, la interfaz se actualiza automáticamente mostrando el enunciado y cuatro cuadrantes coloreados con sus respectivas opciones. Para responder, el alumno debe sostener la mirada hacia el cuadrante deseado (la interfaz lo resaltará visualmente) y ejecutar un parpadeo intencional para confirmar el registro en la base de datos. Adicionalmente, el usuario puede retornar la mirada a la zona neutra (centro) para descansar la vista o revisar su elección sin alterar la selección previa. En paralelo a este proceso, el sistema cliente emite señales periódicas al servidor (*heartbeats*) para indicarle al servidor que mantenga activa la sesión.

(Imagenes de todas las posibles pantallas de cuestionario)

Si la pregunta provino del modo *Invitado*, la sesión concluye inmediatamente mostrando la respuesta enviada. Si se trata de un examen oficial, el sistema muestra todas las preguntas en orden hasta alcanzar la pantalla final, donde se expone el puntaje obtenido, el tiempo transcurrido y la condición de aprobación. Cabe destacar que el profesor posee control total sobre el flujo del examen, pudiendo pausarlo, reanudarlo o finalizarlo remotamente en cualquier momento, lo cual fuerza la actualización inmediata de la pantalla del alumno hacia los estados correspondientes.

Pantallas de acceso restringido

Son aquellas interfaces a las que solo pueden acceder los usuarios registrados mediante el ingreso de sus credenciales (nombre de usuario y contraseña). A todas estas vistas se accede de forma exclusiva superando la validación de la pantalla de login. Si el servidor central (*ESP32*) detecta que un dispositivo intenta conectarse a estas rutas sin poseer un token de sesión activo y válido, el sistema emite una alerta de seguridad y redirige automáticamente al usuario de regreso a la pantalla de inicio de sesión. También posee un límite de tiempo que cierra la sesión automáticamente tras varios minutos de inactividad. En esta categoría se agrupan las pantallas: *Panel*, *Perfil*, *Contactos* y *Cuestionarios*.

La pantalla *Panel* es la interfaz principal que el usuario visualiza inmediatamente tras autenticarse con éxito. Presenta un encabezado con un texto de bienvenida que incluye el nombre y apellido del usuario activo, junto a un botón rojo destinado a cerrar la sesión en cualquier momento. A continuación, se despliegan tres grandes botones de navegación cuyo contenido es dinámico y depende de la jerarquía de la cuenta:

Si el usuario posee el rol de profesor, visualizará las opciones: *Ver cuestionarios*, *Ver datos tutor* y *Editar perfil*. En cambio, si el usuario es un tutor, el botón intermedio se adapta a su necesidad y es reemplazado por la opción *Ver profesores*.

(Imagenes de pantalla panel para ambos usuarios)

El botón *Editar perfil*, común a ambos roles, redirige a la pantalla *Perfil*. Esta vista tiene como objetivo ofrecer al usuario la posibilidad de modificar su información personal en todo momento. Al igual que en el panel principal, conserva el encabezado de bienvenida y el botón de cierre de sesión. Debajo de estos elementos, presenta un formulario estructurado con campos de texto para actualizar los siguientes datos: nombre de usuario, nombre, apellido, contacto y la referencia (materia o vínculo). Los cambios realizados en este segmento se envían a la base de datos al pulsar el botón *Guardar datos*.

(Imagen del primer formulario)

Seguidamente, la interfaz dispone de una sección independiente dedicada a la actualización de credenciales. Para cambiar la contraseña, el sistema solicita ingresar la clave actual seguida de la nueva, omitiendo la necesidad de utilizar la palabra clave maestra, dado que la identidad del usuario ya se encuentra validada por su sesión activa. En la parte inferior, la interfaz provee un botón rojo diseñado para eliminar definitivamente la cuenta del usuario activo. Como medida de seguridad, al accionar este botón el navegador despliega un cuadro de diálogo de confirmación antes de ejecutar el borrado en la base de datos. Finalmente, existe un botón de navegación para regresar al panel anterior, el cual cuenta con una validación interna que advierte al usuario si intenta abandonar la vista existiendo modificaciones sin guardar.

(Imagen del resto)

Retornando al menú principal, el botón central dirige a la pantalla *Contactos*, cuya interfaz y funcionalidad son dinámicas y se adaptan al rol del usuario activo en la sesión. Si el usuario posee el rol de profesor y accede mediante la opción *Ver datos tutor*, visualizará una pantalla encabezada por el título *Datos del Tutor*. Debajo, se despliega una tarjeta informativa que contiene los datos básicos del tutor asociado al dispositivo del alumno: nombre, apellido, vínculo y contacto. Si la base de datos no registra ningún tutor, la interfaz oculta la tarjeta vacía y advierte al usuario mediante un mensaje de texto indicando la ausencia de este perfil. Cabe mencionar que cada tutor debe ser único por cada dispositivo, por lo que se debería mostrar siempre una única tarjeta en caso de haberlo.

(Imagen de ver tutores)

Por el contrario, si el usuario autenticado es un tutor e ingresa mediante el botón *Ver profesores*, el encabezado cambia a *Profesores* y la vista presenta una lista dinámica. Esta lista renderiza, en formato de tarjetas individuales, la información de todos los docentes registrados en el dispositivo, detallando para cada uno: Materia, nombre, apellido y contacto. Asimismo, cada tarjeta incorpora un botón de acción destructiva que permite al tutor eliminar manualmente a cualquier profesor del sistema. Al accionar este comando, la interfaz exige una confirmación explícita antes de ejecutar el borrado definitivo en la base de datos (acción que, por diseño del servidor, elimina en cascada los cuestionarios asociados a dicho docente). Al igual que en la vista anterior, si no existen profesores registrados, se notifica la situación con un breve texto en pantalla.

(Imagen de ver profesores)

Finalmente cabe destacar que, a diferencia del panel principal, esta interfaz prescinde del encabezado de bienvenida y del botón de cierre de sesión, priorizando una visualización limpia que ofrece únicamente un botón inferior denominado *Volver* para retornar de forma segura al menú anterior.

La pantalla *Cuestionarios*, accesible a través del panel principal, constituye el núcleo de la gestión académica del dispositivo, presentando múltiples variaciones funcionales dependiendo de la jerarquía del usuario. Para el profesor, esta pantalla actúa como el motor integral de las evaluaciones: permite crearlas, modificarlas, controlar su ejecución en tiempo real, revisar los

resultados y eliminarlas. La vista inicial consiste en un listado dinámico de todos los cuestionarios asociados a su materia. Cada elemento de la lista expone datos clave: título, cantidad de preguntas, puntaje obtenido, fecha de inicio y su condición final (aprobado o desaprobado). El control de cada módulo se gestiona a través de cuatro botones, los cuales se habilitan o deshabilitan respondiendo a la máquina de estados del servidor:

- Estado **PENDIENTE** (Previo a la ejecución): Se habilitan los botones superiores de acción destructiva y modificadora (*Editar* y *Eliminar*). En el bloque inferior, se habilita el botón *INICIAR* (cuyo accionamiento transiciona el examen al estado *En Progreso*) y se mantiene bloqueado el botón *FINALIZAR*.
- Estado **EN PROGRESO** (Alumno respondiendo): Para garantizar la integridad de los datos, el sistema bloquea inmediatamente los botones de edición y eliminación. El botón de inicio asume la función de *PAUSAR* y se habilita el botón de *FINALIZAR* como mecanismo de aborto seguro en caso de contingencias.
- Estado **PAUSADO**: Los controles superiores de edición permanecen bloqueados y el botón de finalización sigue activo. El botón de estado muta a *REANUDAR*, permitiendo al alumno continuar el examen desde la última pregunta contestada.
- Estado **FINALIZADO**: Los botones de edición y finalización se desactivan. Se rehabilita la capacidad de eliminar el cuestionario y el botón de estado principal se transforma en "REVISAR", redirigiendo al profesor a la vista de revisión.

(Imagen de pantalla cuestionarios de profesor)

Por encima de este listado, un botón etiquetado como "Nuevo +" da acceso a la interfaz de creación y edición. Esta vista presenta una plantilla estructurada encabezada por el título del cuestionario (editable de forma interactiva). A continuación, se despliegan los bloques de las preguntas, los cuales permiten ingresar la incógnita y configurar dinámicamente entre dos y cuatro opciones posibles.

A nivel de configuración, el docente puede seleccionar la respuesta correcta mediante un botón de selección, establecer el puntaje a sumar por acierto y definir el puntaje a restar por error. Para optimizar la carga de datos, la interfaz incluye herramientas de diseño que permiten replicar un mismo esquema de puntajes a todo el examen (*Aplicar a todas*), reordenar la posición de las preguntas en la lista mediante flechas direccionales o eliminar preguntas enteras. En el margen inferior está disponible un campo numérico para establecer el umbral de aprobación y los tres comandos de cierre: *Añadir Pregunta*, *Guardar Cuestionario* (que empaqueta los datos y los envía al ESP32) y *Volver sin guardar*.

(Imágenes de la pantalla del editor)

Retornando a la vista del listado, cuando un cuestionario finalizado es sometido a revisión (mediante el botón *REVISAR*), el sistema despliega una vista analítica. Esta pantalla detalla el tiempo total empleado, el puntaje final, la fecha de inicio, la condición de aprobación, un desglose pregunta por pregunta indicando la opción elegida por el alumno, la opción correcta (si hubo equivocación) y el impacto numérico de dicha respuesta en el puntaje total.

(Imágenes de la vista de revisión)

Por otra parte, si el usuario que accede a la pantalla *Cuestionarios* posee el rol de tutor, la interfaz restringe severamente sus privilegios. El listado mostrará los cuestionarios históricos creados por la totalidad de los profesores, indicando adicionalmente a qué materia pertenece cada uno. Todos los controles de edición, eliminación y alteración de estados se suprimen gráficamente. El tutor únicamente cuenta con el botón de *REVISAR* para consultar las analíticas, el cual

permanecerá bloqueado hasta que el sistema registre que el cuestionario ha alcanzado el estado *FINALIZADO*.

(Imagen de la pantalla cuestionarios del tutor y otra de la pantalla de revisión)

Núcleos de la aplicación

A diferencia de las interfaces visuales descritas anteriormente, esta categoría abarca los archivos y scripts estructurales que operan en segundo plano. No poseen una representación gráfica directa para el usuario, pero constituyen el motor lógico que sostiene la arquitectura de *Aplicación de Página Única* (SPA, por sus siglas en inglés) y gestionan la comunicación con el servidor central.

(Imagen de los archivos nucleo en el visual studio)

El punto de entrada (**index.html** y **main.jsx**): en el paradigma SPA, el servidor *ESP32* aloja y transmite un único documento *HTML* (*index.html*) que actúa como un lienzo en blanco. El archivo *main.jsx* es el script principal de inicialización. Su única y vital función es inyectar la librería *React* dentro de ese documento HTML vacío, montar el *Virtual DOM* (Modelo de Objetos del Documento Virtual) y comenzar a renderizar los componentes de la interfaz a la velocidad del procesador del cliente, evitando que la página web deba recargarse al navegar. Como detalle adicional, el archivo *index.html* también se encarga de descargar el script de *OpenCV* (*opencv.js*, almacenado en la tarjeta microSD) para que esté disponible globalmente en la aplicación.

El gestor de rutas (**App.jsx**): este componente actúa como el controlador de tráfico del sistema. Su función es evaluar continuamente la URL actual del navegador y decidir qué pantalla visual debe renderizarse en cada momento. Además, gestiona la lógica de protección de rutas: antes de permitir que un componente de acceso restringido se monte en pantalla (como el panel o los cuestionarios), *App.jsx* verifica la memoria de sesión (*sessionStorage*) en busca de un token de autorización válido. Si las credenciales no existen o caducaron, bloquea la navegación y redirige el flujo forzosamente hacia la pantalla de *Login*. Este archivo también implementa un monitor de sesión que vigila la inactividad del usuario (10 minutos) y realiza peticiones periódicas (*/api/auth/heartbeat* cada 5 segundos) para asegurar que el token sigue siendo válido en el servidor. Si la validación falla, ejecuta la función *handleLogout*, la cual limpia las credenciales y redirige a la pantalla de inicio de sesión para proteger las herramientas administrativas.

La capa de servicios (**services/api.js**): Para mantener una arquitectura de software escalable y ordenada, el sistema aísla todas las comunicaciones de red en un módulo de servicios independiente. Este script centraliza todas las peticiones asíncronas (HTTP y REST) dirigidas al backend alojado en el *ESP32*. Su existencia permite que los componentes gráficos soliciten datos (como el estado de un examen o la lista de profesores) invocando funciones simples, mientras este archivo se encarga de la complejidad de estructurar las cabeceras de red, adjuntar los tokens de seguridad y parsear las respuestas JSON provenientes del microcontrolador.

Compilación y despliegue (**dist/**): Cabe destacar que el código fuente desarrollado en *React* no puede ser interpretado nativamente por los navegadores ni almacenado de forma eficiente en un microcontrolador. Por este motivo, el proyecto utiliza una herramienta de construcción (en este caso, *Vite*) que compila, minimiza y empaqueta todo el código fuente en un conjunto de archivos estáticos altamente optimizados (ubicados en el directorio *dist/*). Son exclusivamente estos archivos livianos los que finalmente se transfieren a la memoria microSD del *ESP32* principal para ser servidos a los usuarios.

3.5.2. Backend (Naty)

3.6. Interfaz de usuario táctil

Con el fin de aumentar y mejorar la accesibilidad al momento de utilizar el dispositivo, se optó por incorporar una interfaz visual directamente en el microcontrolador servidor. En el mercado existen diversas variantes de módulos *ESP32* que difieren según sus periféricos integrados. Uno de ellos es el modelo *ESP32-2432S028*, conocido coloquialmente como *ESP32-CYD* (*Cheap Yellow Display*). Este hardware incorpora el mismo procesador utilizado a lo largo de este proyecto, con el agregado particular de una pantalla LED táctil integrada.

(Imagen de CYD)

Este componente permite renderizar elementos interactivos como textos, imágenes, botones y teclados virtuales, mejorando significativamente la experiencia de usuario de forma autónoma (sin depender de un dispositivo externo). Para gestionar y dibujar esta interfaz gráfica, el sistema emplea dos librerías fundamentales:

- **TFT_eSPI**: Otorga los controladores (drivers) de bajo nivel necesarios para la comunicación por hardware con el display.
- **LVGL**: Es una librería gráfica de alto nivel que permite renderizar y gestionar la lógica de los elementos visuales y las interacciones táctiles del usuario.

A nivel arquitectónico, la interfaz se divide en dos grandes bloques lógicos: un contenedor deslizable (*TileView*) que agrupa tres vistas de uso continuo, y una cuarta vista aislada destinada a la configuración inicial del sistema. Toda la información necesaria la consigue mediante comandos sencillos que comunican el backend con el frontend.

Vista 1 - Bienvenida

Es la primera interfaz que visualiza el usuario al encender el dispositivo ya configurado. Su diseño es minimalista y consta de un texto de bienvenida acompañado de un código QR. Al escanear este código con cualquier teléfono móvil, el usuario es redirigido automáticamente a la URL local del servidor (*levi.local*), simplificando el acceso y evitando la introducción manual de direcciones IP.

(Imagen vista 1)

Vista 2 - Estado del sistema

Al deslizar la pantalla hacia la derecha, se accede a un panel de monitoreo. Su función es exponer en tiempo real los datos críticos sobre el funcionamiento de la red local y los nodos conectados. La información expuesta incluye:

- Nombre del alumno propietario del dispositivo.
- Nombre del usuario administrativo (profesor o tutor) con sesión activa.
- Estado del módulo de captura visual (indicando si la cámara está *Transmitiendo* o *Desconectada*).
- Cantidad total de clientes conectados al servidor.

En condiciones normales de uso deberían haber tres dispositivos conectados: el alumno, el profesor y la cámara.

(Imagen vista 2, varias posibilidades)

Vista 3 - Examen

Es la última pantalla del flujo continuo. Su propósito es brindar un seguimiento de las evaluaciones activas, permitiendo al docente (u observador externo) conocer el progreso del alumno directamente desde la carcasa del dispositivo. Cuando no hay cuestionario activo, esta pantalla está en modo de espera, con un texto azul centrado que indica *'Esperando cuestionario'*

Cuando hay un cuestionario activo, la pantalla cambia mostrando su título y el progreso actual del alumno.

Vista 4 - Inicialización

A diferencia de las anteriores, esta vista se ejecuta de manera exclusiva durante el primer arranque del equipo (o tras un formateo de fábrica). Su objetivo es configurar las credenciales permanentes del sistema operativo. Mediante el uso de un teclado táctil virtual desplegable, el administrador debe ingresar tres parámetros fundamentales: el nombre del alumno, la palabra clave maestra para profesores y la palabra clave maestra para tutores. Estas credenciales actúan como la raíz de seguridad del dispositivo. Una vez completados los campos, el usuario debe oprimir el botón de guardado, el cual almacena las variables de forma persistente en la memoria no volátil (NVS) del microcontrolador y fuerza un reinicio automático del sistema, cargando finalmente la vista de bienvenida convencional.

(Vista pantalla de booteo)

3.7. Alimentación

Entre las premisas de este trabajo se estipuló que el dispositivo final debe ser inalámbrico y autosuficiente en términos de consumo energético al momento de utilizarlo. Siguiendo estos lineamientos, se opta por no alimentarlo utilizando cables y entradas USB-C o MicroUSB que se incluyen en los modelos comerciales de los microcontroladores utilizados, sino que se diseñará su propio sistema de alimentación utilizando fuentes de energía independientes de cualquier dispositivo externo al momento de uso. La solución general planteada para este problema recae en el uso de las baterías de litio, una de las fuentes de energía inalámbrica más comunes en los dispositivos electrónicos de bajo consumo. A pesar de su precio relativamente elevado a comparación de las pilas comunes, tienen la particularidad de que son recargables, evitando así que el usuario tenga que comprar repuestos en el momento en que se agote la energía.

La alimentación a través de estas pilas permitirá un uso prolongado del dispositivo antes de necesitar una recarga y al mismo tiempo evita la necesidad de comprar pilas comunes para cada período de uso. La energía suministrada será moldeada por otros componentes electrónicos para obtener la salida continua y estable que requieren estos microcontroladores para funcionar correctamente. Cada placa (emisora de la gorra y CYD receptora) tendrá su propio circuito de alimentación independiente de la otra, para así evitar enlazar ambas partes del proyecto mediante un cable y poder mover a la CYD con la interfaz libremente. En esencia, ambos circuitos serán exactamente iguales, pues se trata con dos microcontroladores que, para su uso en este contexto, requieren de la misma señal de alimentación.

Teniendo seleccionada la fuente de energía, se presentan otros dos problemas. Se requiere una forma para recargar las pilas de manera sencilla, y además un método para estabilizar la señal de alimentación. Esto último se debe a que los *ESP32* requieren una alimentación de 3,3 V o, en su defecto, de 5 V lo más estables posible, ya que una mala alimentación puede provocar mal funcionamiento, reinicios inesperados o destrucción del hardware.

3.7.1. Soluciones propuestas

1- Alimentación directa + Regulador lineal de tensión + Cargador de pilas externo

Consiste en alimentar el microcontrolador directamente desde la pila de litio a través de un portapilas. A esto se le suma un regulador de tensión lineal de baja caída (LDO), como el *AMS1777*, para ajustar la tensión a 3,3 V estables. Para recargar las pilas se emplearía un cargador comercial externo, que requiere extraer físicamente la pila del dispositivo para colocarla en la ranura y conectarla a la red.

(Imagen del LDO)

Ventajas de esta solución:

- Mantenimiento práctico: Los cargadores comerciales son componentes estándar y fáciles de conseguir en el mercado.
- Optimización de espacio: Al requerir únicamente un portapilas, el regulador lineal y un par de capacitores de filtrado, se reduce considerablemente el espacio ocupado en una PCB.

Desventajas de esta solución:

- Ergonomía y desgaste mecánico: Resulta impráctico para el usuario el tener que retirar la pila del dispositivo cada vez que se quiera recargar, además de que esto puede provocar desgaste mecánico con el tiempo.
- Descarga de la pila: Una pila de litio estándar (como la *18650*) de 3,7 V no entrega este valor continuo de tensión a lo largo del tiempo, sino que disminuye progresivamente hasta llegar a un piso mínimo de unos 3,0 V. Este valor es menor a los 3,3 V que necesita el microcontrolador, lo que significa que el dispositivo dejaría de funcionar mucho antes de que la batería se quede sin energía para dar. Se requiere hardware adicional para aumentar el valor de tensión cuando sea necesario, y agregarlo invalida la ventaja de la optimización de espacio.
- Dropout: La función del regulador es estabilizar la tensión en 3,3 V. Sin embargo, estos integrados operan con transistores MOSFET en zona activa (es decir, actuando como resistencias variables), lo que implica inevitablemente que habrá una caída de voltaje entre la entrada y la salida. Esta caída se llama *voltaje de dropout*, lo que significa que para tener 3,3 V estables a la salida, necesitaremos una tensión continua mayor que ese valor en la entrada, lo que reduce aún más el tiempo en el que se puede usar el dispositivo antes de que la pila se quede sin energía.
- Picos de corriente: El consumo de corriente de una *ESP32* es dinámico. Hay ocasiones, como al encender el Wi-Fi, en las que el microcontrolador exige grandes transitorios o picos de corriente de hasta 500 mA. Por ley de ohm, al exigir más corriente al regulador, la caída de tensión de dropout por la resistencia del silicio se maximizará. Esto agrava el problema anteriormente mencionado, provocando caídas bruscas en la tensión de alimentación que disparan el detector de bajo voltaje (*brownout detector*) del microcontrolador.

2- Placa de alimentación (Módulo TP4056 + Módulo MT3608)

El módulo *TP4056* es un circuito integrado utilizado para recargar baterías como las que se usarán en este proyecto, mientras que el *MT3608* es un módulo elevador de voltaje (boost) que entrega a la salida una señal muy estable del valor elegido por el diseñador. Si se interconectan ambos módulos se formaría un conjunto que cumpliría los requisitos de recarga y regulación, y se podría alimentar al *ESP32* desde el pin de 5 voltios.

(Imagen de los módulos en cuestión)

Ventajas:

- Facilidad de mantenimiento: Al ser dos módulos separados, las fallas puntuales pueden repararse fácilmente utilizando un repuesto del módulo necesario sin alterar el sistema completo.
- Carga térmica: Al ser dos módulos diferentes que se encargan de distintas tareas, la temperatura que alcance cada uno será independiente del otro y se distribuirá uniformemente a lo largo de la PCB.
- Sin dropout ni picos de corriente: El módulo *MT3608* es un regulador switching, que utiliza transistores MOSFET en zona de corte y saturación, por lo que no hay caídas apreciables de tensión entre la entrada y la salida. A su vez, los picos de corriente pueden ser abastecidos por el ajuste automático del ciclo de trabajo del switch.
- Doble regulación de tensión: Al alimentar el *ESP32* a través del pin de 5 V, se utiliza el regulador integrado en la placa para reducirlos a 3,3 V aún más estables.

Desventajas:

- Mantenimiento impráctico: A pesar de que las reparaciones son más sencillas y baratas, los módulos mencionados sólo pueden conseguirse en casas especializadas en electrónica, a diferencia del cargador estándar.
- Menor optimización del espacio: Al tener dos módulos distintos, se requiere una PCB más espaciosa para contenerlos.
- Pérdida de eficiencia: La ventaja de la doble regulación implica inevitablemente una pérdida de eficiencia. La energía sufre dos conversiones: de la pila a los 5 V y de los 5 V a los 3,3 V en el regulador interno del *ESP32*, lo que aumenta la disipación térmica.

3- Placa de alimentación con un regulador Buck-Boost

Se podría pensar que para evitar esa pérdida de eficiencia con el *MT3608* y alimentar al microcontrolador con 3,3 V se pudiera utilizar un regulador Buck-Boost (como el *XL6009*) en lugar del elevador Boost para alimentarlo a 5 V. Pero la ventaja de la eficiencia contrasta con el aumento del costo y la complejidad del circuito integrado. Más adelante se realizará un análisis de mercado para comprobar este punto.

(Imagen del BuckBoost)

4- Integrado Power Bank todo en uno El integrado *IP5306* cumple la misma función que la placa de alimentación utilizando un solo chip integrado que realiza internamente el proceso de carga y de regulación de tensión.

(Imagen del IP5306)

Ventajas:

- Optimización del espacio en PCB al ser un solo módulo.
- Regulación conmutada: Al igual que los elevadores y reductores, funcionan con transistores en zona de corte y saturación, evitando el problema del dropout y los picos de corriente.
- Carga y uso simultáneo.

Desventajas:

- Mantenimiento impráctico, por la dificultad agregada de conseguir los repuestos.
- Pérdida de eficiencia por usar doble regulación.
- Carga térmica: Al ser un solo chip el que hace la tarea de carga de batería y regulación de voltaje, todo el calor disipado se acumulará en un único punto del dispositivo, lo que influirá negativamente en el funcionamiento del mismo.
- No es modular: Si se produjera una falla en cualquier parte del sistema, inevitablemente habría que comprar el repuesto completo del chip, lo cual es más costoso que comprar los repuestos separados de los módulos anteriormente vistos.
- Ripple en la señal de salida: Al estar diseñados para cargar celulares conectándose a un PMIC interno, los chips *IP5306* conmutan a mucha menor frecuencia (entre 300 y 500 kHz), que el módulo *MT3608*, por ejemplo. Además, carece de filtros adecuados a la salida, lo que provoca que este rizado sea más difícil de eliminar. Este ripple es sumamente indeseable para alimentar un servidor Wi-Fi y una cámara que debe transmitir en vivo.

Para solucionar el problema del ripple, se podría utilizar directamente un PMIC industrial, pero su elevado costo y la complejidad de soldadura de sus encapsulados lo vuelven inviable para este prototipo.

Análisis de mercado

(Tabla con las comparaciones de precios)

3.7.2. Descripción del sistema elegido

El primer circuito integrado que utilizaremos es el chip *TP4056*, cuyo datasheet lo describe como un cargador lineal de corriente constante/tensión constante completo para baterías Li-ion de una sola celda. Este chip toma energía de una fuente externa y la adapta para recargar la batería. La energía que proveen estas baterías de larga duración suele ser de 3,7 voltios. Por lo tanto, para adecuar y estabilizar este nivel de tensión se utiliza otro circuito integrado fundamental: el *MT3608*; el cual, en términos sencillos, se trata de un circuito elevador boost que en la salida entrega una señal continua y estable según el ajuste que realice el diseñador. Esa salida irá directamente conectada a la entrada de 5 voltios de la placa *ESP32*.

El funcionamiento en detalle de estos integrados y sus módulos comerciales serán descritos en las siguientes secciones.

3.7.3. Recarga y protección

Como se expuso con anterioridad, el chip *TP4056* sirve para cargar una única celda de baterías de litio, y lo hace en un ciclo de dos fases.

En primera instancia carga la batería a corriente constante, es decir, le inyecta un valor fijo de corriente elegida por el diseñador para que la carga comience siendo rápida. Al mismo tiempo, el chip mide constantemente el voltaje de la pila, y cuando llega a los 4,2 V comienza la fase de carga con voltaje constante, y permite a la propia pila que absorba la corriente necesaria para no destruirse. Si toda la carga fuera a corriente constante, se le inyectaría a la pila más energía de la que puede soportar y acabaría destruyéndose. Por el contrario, si toda la carga fuese a voltaje constante, el proceso llevaría demasiado tiempo, lo cual es poco práctico para la experiencia de usuario. Es por eso que este integrado oscila automáticamente entre esos dos modos.

Por otra parte, el *TP4056* está diseñado para detener la carga en la fase de voltaje constante, cuando la corriente absorbida por la pila sea inferior a una décima del valor programado por el usuario, lo cual evita sobrecargar la pila.

Para resumir, sus características más relevantes son:

- Corriente de carga programable hasta 1000 mA.
- Operación dinámica entre corriente constante y voltaje constante.
- Carga hasta un valor fijo de 4,2 V.
- Fin de carga automático.
- UVLO.
- Umbral de carga lenta.
- Arranque suave de corriente.

UVLO (*Under Voltage Lockout* / Bloqueo por bajo voltaje): Es una funcionalidad del chip que permite que cuando se introduzca una alimentación mala o inestable, la carga se bloquee para evitar que el proceso dañe la pila o los circuitos.

Umbral de carga lenta: Para evitar sobrecargar una pila en mal estado (con voltaje inferior a 3,0), el *TP4056* entra en modo *carga lenta*, enviando solo el 10 % de la corriente programada hasta que supere el umbral. Luego de eso la carga procede con normalidad.

Arranque suave de corriente: Es una medida de protección a la fuente externa. Garantiza que al enchufar el circuito a la misma, la absorción de corriente no sea tan brusca, sino que aumente gradualmente con el tiempo para evitar daños.

Su diagrama de pines es el siguiente:

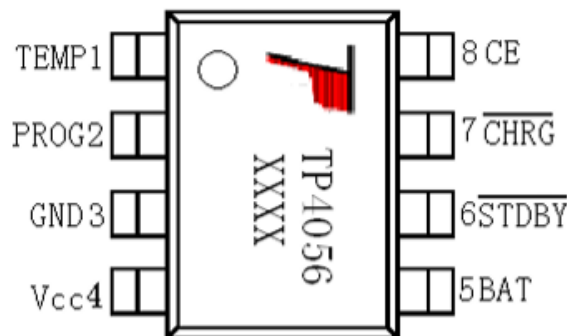


Figura 3.19: Diagrama de pines del módulo *TP4056*

1. **TEMP:** Entrada de sensor de temperatura. Sirve para monitorear la temperatura de la pila al momento de la carga mediante un termistor externo. Debido al uso que se le va a dar en este proyecto y a la presencia de otros circuitos de protección, se opta por prescindir de su uso.
2. **PROG:** Entrada de programación de corriente de carga. A este pin se le debe conectar una resistencia denotada como *RPROG* para establecer el valor de la corriente de carga. El valor esta se calcula con la siguiente fórmula:

$$R_{prog} = 1200 * I_{bat} \quad (3.1)$$

Donde I_{bat} es la corriente en miliamperios elegida por el diseñador, con un valor máximo de 1000 mA.

3. GND: Tensión de referencia.
4. VCC: Entrada de alimentación externa.
5. BAT: Salida que se conecta al terminal positivo de la pila. Le brinda la corriente necesaria y regula el voltaje a 4,2 V.
6. STDBY: Salida en alta impedancia que se vuelve nula cuando la carga está completa.
7. CHRG: Salida en alta impedancia que se vuelve nula mientras la pila se está cargando.
8. CE: Habilitación del chip. Conectándolo a un 0 lógico se bloquean todas las funcionalidades del chip (Modo *shutdown*).

En el siguiente gráfico se ilustran las fases de corriente constante y voltaje constante descritas anteriormente:

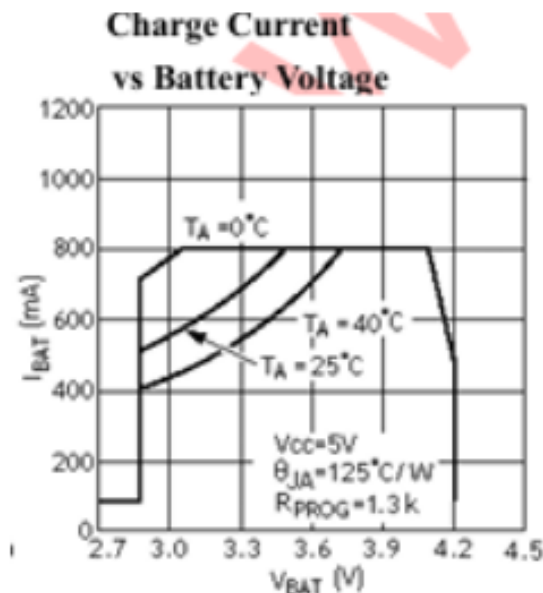


Figura 3.20: Corriente vs voltaje de carga

Con un voltaje menor a 4,2 V, se programó en ese caso una corriente de 800 mA. El ciclo comienza con un aumento gradual de la corriente hasta llegar a ese valor programado. El voltaje mientras tanto va aumentando, y al llegar a los 4,2 V, la corriente disminuye con ese mismo valor hasta finalizar la carga.

Se explica a continuación un circuito de uso típico con este chip, correspondiente al módulo que se encuentra comercialmente en las casas de electrónica y se utiliza en este proyecto:

Lo primero que se puede notar es un puente entre la entrada CE y VCC, esto es debido a que para este uso práctico no se requiere deshabilitar el chip manualmente. Se observa también un capacitor C4 utilizado para drenar el ruido de la entrada

En las salidas CHRG y STDBY están conectados los cátodos de dos diodos LED. Uno verde para STDBY y uno rojo para el CHRG. Estos son LEDs indicadores que informan al usuario en qué estado se encuentra la carga.

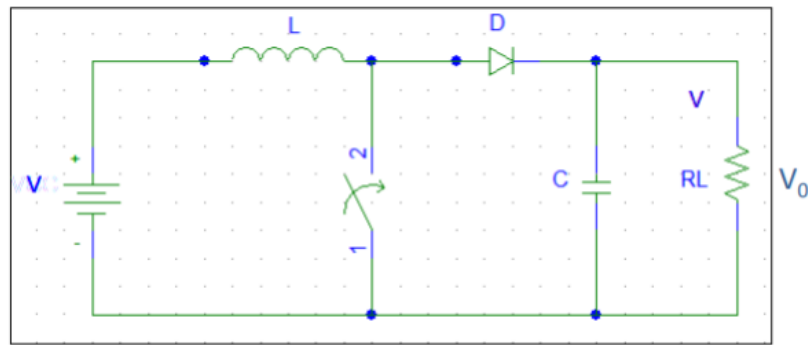


Figura 3.22: Circuito elevador Boost

Este circuito es el típico que se utiliza para las fuentes de tensión tipo *switching*. En este caso particular, la tensión continua de la entrada es la salida de la pila de Litio, que oscila entre 3,0 y 3,7 V. Esta señal pasa por un conjunto de una bobina, un diodo, un capacitor, y un switch que se abre y cierra intermitentemente para brindar a la salida el valor deseado por el diseñador. Para ello se aprovecha los transitorios de la bobina en los primeros instantes en que esta llave se abre o cierra.

Cuando la llave se cierra, la bobina comienza a cargarse con la energía de la pila, y la tensión a la salida es igual a la tensión previamente almacenada en el capacitor, el cual no puede descargarse en el cortocircuito debido a la presencia del diodo D, que se polariza en inversa y causa que toda la corriente vaya desde el capacitor hacia la carga.

Cuando la llave se abre, la bobina se descarga y cambia su polaridad, enviando corriente a través del diodo, cargando el capacitor y otorgando un valor de tensión a la salida igual a la suma de las tensiones de entrada y de la bobina.

La tensión de la bobina cuando la llave se abre, y por consecuencia la tensión del capacitor cuando la llave se cierra, dependen de los tiempos del llaveo, más específicamente del ciclo de trabajo o *duty cycle*:

$$DutyCycle = ton / (ton + toff) \quad (3.2)$$

Este ciclo de trabajo es regulado por el propio *MT3608*, el cual es la llave física de este circuito (un MOSFET), sumado a un circuito interno que monitorea la tensión de salida, la compara con una tensión de setpoint elegida por el diseñador y controla los tiempos de llaveo según requiera el caso. Si la tensión de salida cae, el ciclo de trabajo será mayor, mientras que si tenemos un exceso de tensión, este ciclo de trabajo será menor.

Las características más importantes de este integrado son:

- Switch: MOSFET de potencia de 80 mΩ
- Tensión de entrada permitida: Entre 2 y 24 V
- Tensión de salida ajustable de hasta 28 V
- Frecuencia de llaveo: 1.2 MHz
- Límite de corriente a través de la llave de 4 A
- Funcionamiento con modulación de ancho de pulso (PWM)

Su circuito interno es el siguiente:

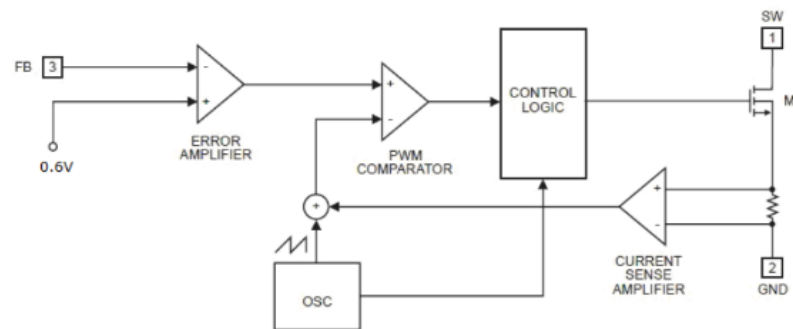


Figura 3.23: Circuito interno del chip *MT3608*

El transistor M1 es el switch que se observa en el diagrama del circuito boost, que se manipula mediante un bloque lógico de control conectado a su entrada de compuerta.

El control comienza con el amplificador de error, que compara la entrada de feedback del chip (una muestra de la tensión de salida final), con una señal de 0,6 voltios muy estable. A la salida de este amplificador tenemos una señal proporcional al error de la tensión en la carga. El diseñador debe procurar que cuando la tensión en la carga se encuentre en su valor óptimo, la entrada de feedback debe ser igual a esos 0,6 voltios, para que en esa situación la salida sea constante y no afecte a la modulación.

Cuando la salida es mayor a su valor esperado ($V_{FB} > V_{REF}$), el comparador de error reacciona disminuyendo la señal de error, mientras que cuando la salida es menor ($V_{FB} < V_{REF}$), la señal de error aumenta. Finalmente, si ambas entradas son iguales ($V_{FB} = V_{REF}$), el error será una señal constante, como se mencionó anteriormente. Todo lo anterior se refleja en la siguiente imagen:

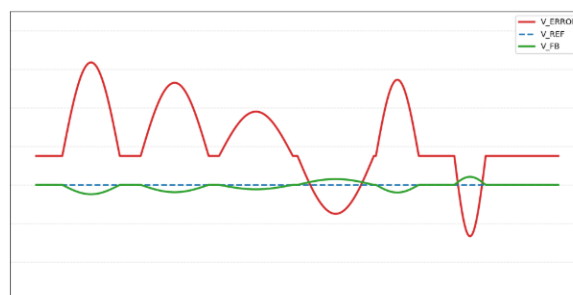


Figura 3.24: Gráfico explicativo del comportamiento del amplificador de error

Este valor de error ingresa en el amplificador comparador PWM junto a una señal diente de sierra generada por un oscilador, para que a su salida se obtenga la señal de control que finalmente manipule los estados del transistor M1. El mismo oscilador diente de sierra es quien activa el pulso de la compuerta, cerrando así la llave y permitiendo que la bobina se cargue. Mientras la tensión de error supere a la señal del oscilador, el pulso se mantiene activo. Cuando la señal diente de sierra supera a la señal de error, el pulso se desactiva abriendo la llave y dejando que la bobina se descargue hasta el próximo ciclo.

Si la bobina pasa más tiempo cargándose (ciclo de trabajo grande), la tensión que genera al momento de descargarse aumentará junto con la tensión de salida, y si pasa más tiempo

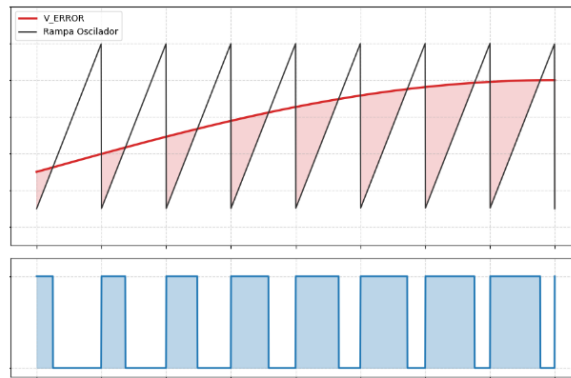


Figura 3.25: Modulación por ancho de pulso (PWM)

descargándose (ciclo de trabajo pequeño), la tensión de descarga será menor disminuyendo la salida final.

De todo este análisis se puede inferir la siguiente conclusión: Si la tensión de salida decae, el ancho del pulso, y por consiguiente la tensión de salida, aumentan. Por el contrario, si ocurre un exceso de voltaje en la carga, tanto el ancho de pulso como la tensión de salida disminuyen.

Por último, en el diagrama de bloques se observa un último amplificador que cumple dos funciones importantes: el sensor de corriente. Por un lado, actúa como sistema de protección, detectando situaciones en las que la corriente a través del switch aumenta a valores peligrosos. Para ello, lee la tensión sobre una resistencia del orden de los miliohmios (para interferir lo menos posible con el circuito original) y, ante un pico indeseado, envía una señal a la lógica de control que abre la llave para evitar la destrucción del circuito. Por otro lado, durante el funcionamiento normal, esta misma lectura de corriente se suma a la señal del oscilador para alimentar al comparador PWM, esto permite que el chip ajuste el ancho de pulso instantáneamente en cada ciclo, garantizando estabilidad ante variaciones rápidas de carga.

Con todo el funcionamiento explicado, podemos pasar a mostrar el diagrama de pines:

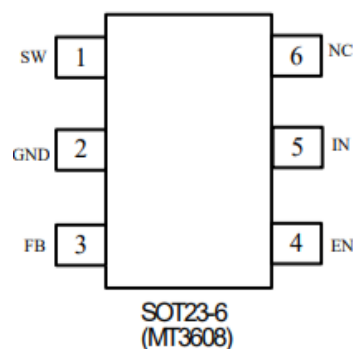


Figura 3.26: Diagrama de pines del módulo MT3608

- SW: Es el pin que conecta al MOSFET M1 con el circuito boost externo (El drenador del transistor).
- GND: Tensión de referencia.
- FB: Tensión de feedback. Es una muestra de la tensión de salida que luego se compara con los 0,6 V internos.
- EN: Habilitación del chip.

- IN: Tensión de entrada.
- No se utiliza.

A continuación se muestran algunos gráficos de funcionamiento típico.

Regulación de carga: Indica la corriente máxima que puede entregar a una carga de determinado valor de tensión (en este caso, con 500 mA es más que suficiente para suplir las exigencias del *ESP32*).

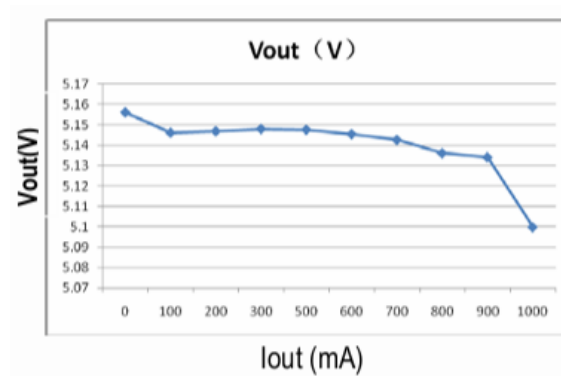


Figura 3.27: Regulación de carga del *MT3608*

Frecuencia de conmutación según tensión de entrada: Vemos que en nuestro caso llegamos a valores de entre 0,9 y 1 Mhz. No es el ideal de 1,2 que se describió anteriormente, pero a fines prácticos sirve para mitigar lo suficiente el ripple de salida.

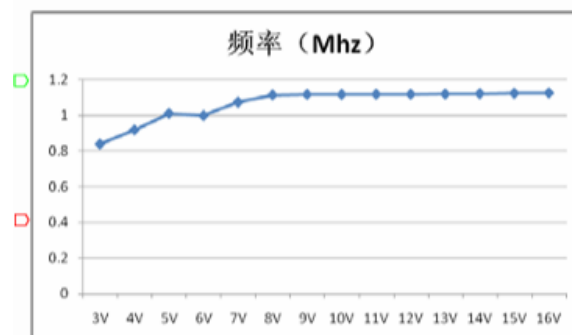
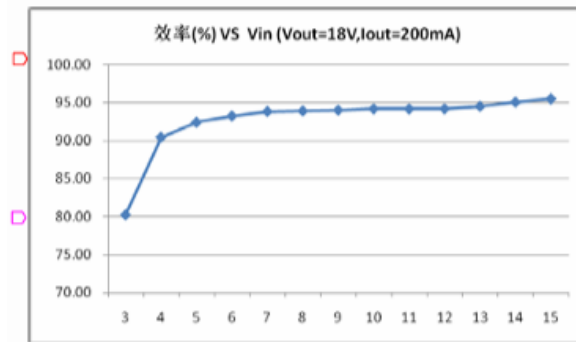
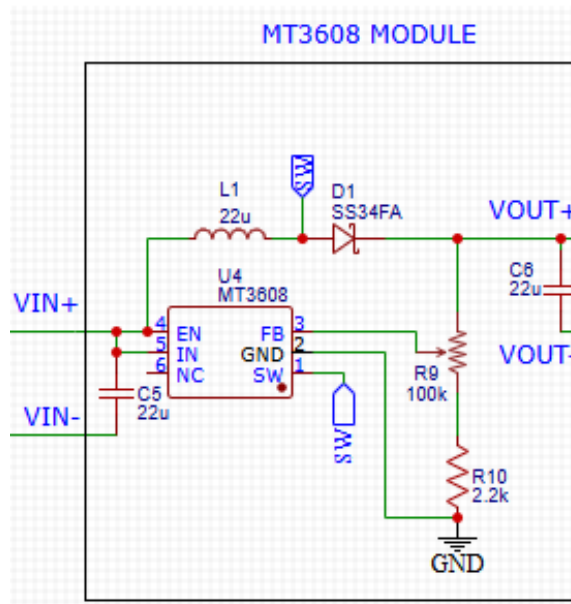


Figura 3.28: Frecuencia vs Tensión de entrada en el chip *MT3608*

Eficiencia vs tensión de entrada: Mientras mayor la tensión de entrada, más eficiente el circuito. En este caso se acerca al 90 %

Figura 3.29: Eficiencia vs Tensión de entrada en el chip *MT3608*

A continuación se muestra el circuito que viene en los módulos *MT3608* comerciales, que incluyen el circuito boost externo:

Figura 3.30: Circuito comercial de los módulos *MT3608*

El circuito comienza con los pines de entrada y habilitación cortocircuitados y conectados a la tensión de 3,7 voltios provenientes de la pila, ya que al igual que con el *TP4056*, no se requiere deshabilitar manualmente el integrado. A su vez, el pin de entrada se conecta a uno de los terminales de la bobina de 22 μ H, para cargarla en los momentos en que la llave se encuentra cerrada.

El pin de switch (el drenador del MOSFET), se conecta al otro terminal de la bobina y al ánodo de un diodo de alta velocidad, como indica el modelo del circuito boost expuesto anteriormente.

Finalmente, el pin de feedback se conecta a un potenciómetro de 100 $k\Omega$ en serie con una resistencia de 2,2 $k\Omega$. El valor de este potenciómetro se ajusta con un tornillo externo para seleccionar el valor de tensión a la salida (dado por la señal obtenida en el amplificador de error). Se colocan esos valores de resistencia para limitar la tensión de salida a un mínimo de 0,6 V y un máximo absoluto de 28 V.

Se observa también el capacitor C6 del circuito boost, un capacitor C5 de desacople, y el pin de GND conectado a la masa del circuito.

Componentes a utilizar en el dispositivo final

Con todo el sistema explicado, se listan a continuación todos los materiales necesarios para construir los circuitos de alimentación:

- 4 Borneras de 2 terminales
- 4 Capacitores cerámicos de 100 nF (para reducir el ruido).
- 2 Baterías de Litio de 3,7 voltios con 3000 mAh de energía.
- 2 Capacitores electrolíticos de 10 uF (para reducir picos de corriente).
- 2 Módulos de carga *TP4056* comerciales (con chips *DW01A* y *FS8205A* incluidos).
- 2 Módulos elevadores *MT3608* comerciales.
- 2 PCBs de las dimensiones adecuadas.
- 2 Resistencias de 1/4 de vatio de 160 ohmios para los diodos LED.
- 2 Switches para encendido del dispositivo.
- 2 Zócalos o portapilas individuales.
- 1 Bornera de 3 terminales.
- 1 Diodo LED azul.
- 1 Diodo LED infrarrojo emisor.
- Zócalos hembra-macho para portar la *ESP32-CAM* en la PCB

3.8. Costo total de desarrollo

(Tabla hecha por naty)

Capítulo 4

Ensayos y resultados

4.1. Montaje

En esta sección se describe y expone el diseño físico final del dispositivo, comenzando exclusivamente por los circuitos de alimentación y posteriormente con el armado final.

4.1.1. Conexiones y PCB

El esquema de conexiones final para el circuito de alimentación del capturador de imágenes es el siguiente:

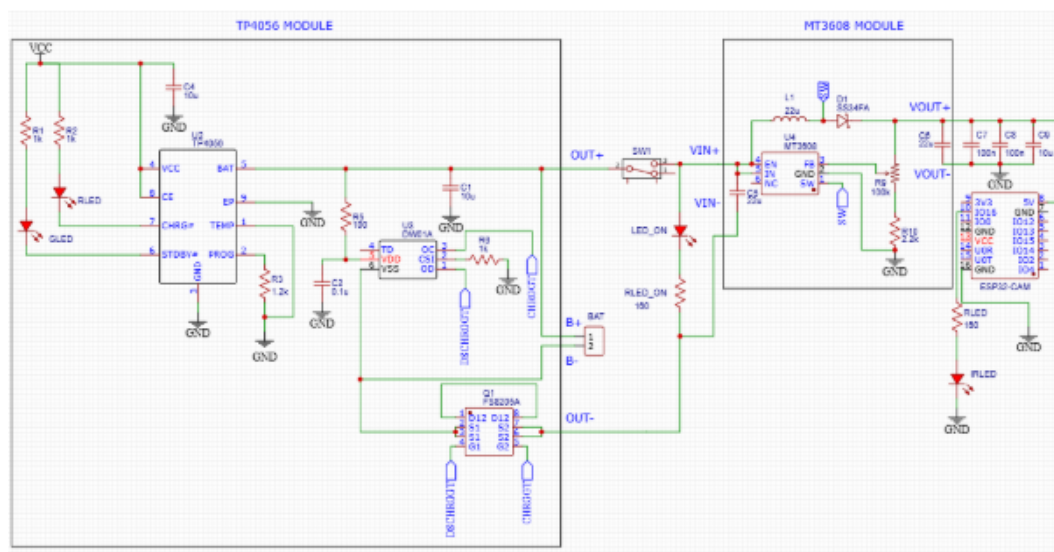


Figura 4.1: Esquema de conexiones para la PCB del circuito capturador

Incluye los módulos *TP4056*, *MT3608*, un switch intermedio entre ellos para desconectar manualmente el dispositivo, y finalmente el zócalo donde se inserta la *ESP32*, al cual se le conecta el LED IR para normalizar la iluminación en el ojo.

Al tener los módulos comerciales, el esquemático final de la PCB se reduce a este simple esquema:

La entrada del *TP4056* se conecta mediante USB-C, por lo que no se incluye en el esquemático de la PCB.

A su vez, el esquema de conexiones de la placa que actuará como servidor web es el siguiente:

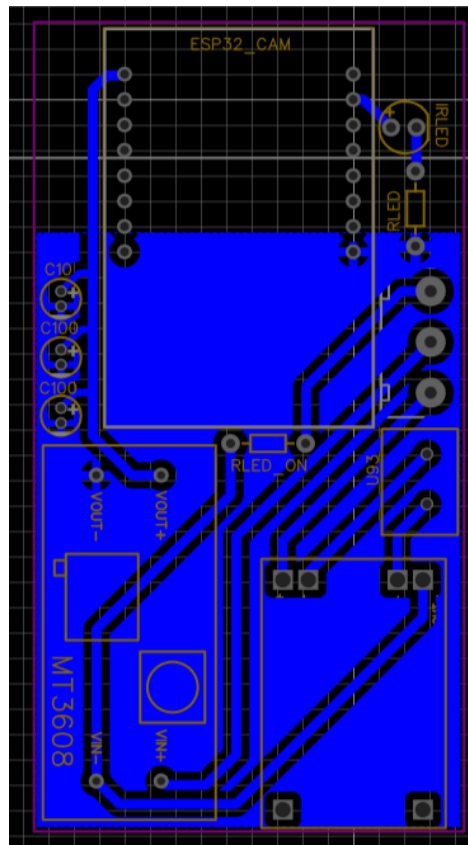


Figura 4.4: Layout de la PCB del circuito captador

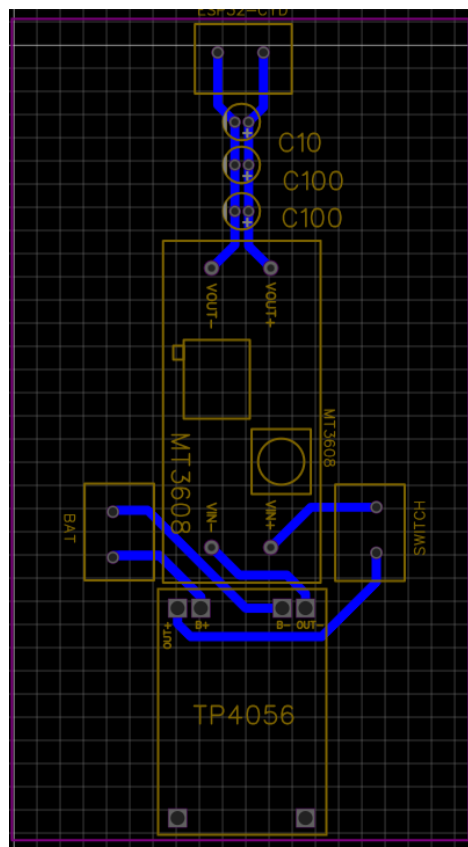


Figura 4.5: Layout de la PCB del circuito servidor

Capítulo 5

Conclusiones

5.1. Resultados obtenidos

Como resultado de este trabajo, se obtuvo un dispositivo de sólido funcionamiento, cómodo, práctico y económico. Si bien no se cumplieron las expectativas iniciales de implementar el seguimiento ocular mediante redes neuronales, se encontró una alternativa matemática que resultó ser mucho más efectiva y eficiente. Este cambio de rumbo fue fundamental para lograr que un hardware tan limitado pudiera cumplir a la perfección su función de actuar como un sistema de evaluación de opción múltiple. A lo largo del desarrollo, muchas de las ideas y procedimientos propuestos al principio tuvieron que ser descartados debido a problemas inevitables de precisión, latencia y disponibilidad de recursos. Es importante destacar la cantidad y variedad de áreas en las que se tuvo que profundizar para llegar a la solución definitiva: programación de sistemas embebidos, desarrollo de aplicaciones web, protocolos de comunicación (como SPI y Wi-Fi), diseño de redes y electrónica básica, entre otros conceptos que ya fueron expuestos a lo largo de este informe. Fue justamente esta mezcla de conceptos y ámbitos tan distintos lo que permitió construir el sistema actual. Se logró crear una solución que gestiona usuarios y sesiones de forma segura, administra cuidadosamente la escasa memoria y potencia del procesador, aprovecha al máximo la comunicación inalámbrica aislada y ofrece interfaces de usuario interactivas. Y lo más destacable: todo esto se consiguió utilizando componentes fácilmente accesibles, económicos y con gran disponibilidad en el mercado. Por todo lo expuesto, aunque el dispositivo final terminó teniendo una funcionalidad técnica distinta a la propuesta originalmente y presenta mejoras a tratar en futuras versiones, el desarrollo se considera un éxito. Especialmente si se tiene en cuenta la premisa social impuesta desde el primer día: se logró fabricar un artefacto de asistencia sumamente versátil, portable e intuitivo para cualquier tipo de usuario.

5.2. Comparación con productos similares

Como bien se puede deducir por lo expuesto en la sección anterior, este dispositivo no busca competir en potencia y velocidad de hardware con los equipos comerciales de detección ocular de gama alta. Al depender de los recursos limitados de un microprocesador sencillo, es lógico que sus especificaciones técnicas sean inferiores. Lo mismo ocurre con el software: al ser un programa embebido en un microcontrolador, no cuenta con un sistema operativo complejo ni con la infraestructura de ciberseguridad necesaria para una comercialización masiva a gran escala. Sin embargo, como contrapartida a estas limitaciones técnicas, el proyecto presenta un punto fuerte fundamental frente a todos sus competidores: su precio y nivel de accesibilidad. Mientras que dispositivos como los *Tobii Glasses* tienen precios que rondan los miles de dólares, el costo total del hardware utilizado para construir este dispositivo fue de apenas unas decenas de dólares. Además, estos productos comerciales rara vez se encuentran disponibles en los mercados locales de nuestra región, lo que suma costos de envío e importación que dificultan aún más su

adquisición. Otra gran ventaja, si lo comparamos con alternativas de software puro como las de *EyeCom Tec*, es que este proyecto cuenta con una interfaz y funcionalidades diseñadas de manera exclusiva para el entorno académico y familiar del estudiante. El sistema fue pensado desde cero para crear y responder preguntas altamente personalizables, brindando comodidad a docentes y usuarios en general que no están familiarizados con tecnologías complejas, y evitando el tener que realizar configuraciones previas que suelen ralentizar el proceso. En resumen, si bien no es un dispositivo pulido para competir en el mercado del hardware de alto rendimiento, cumple con creces su objetivo principal: ofrecer una experiencia de comunicación sencilla, intuitiva y a un precio realmente accesible para cada usuario.

5.3. Trabajo futuro